GRAPHS

OBJECTIVES

- Explain what a graph is
- Compare and contrast different types of graphs and their use cases in the real world
- Implement a graph using an adjacency list
- Traverse through a graph using BFS and DFS
- Compare and contrast graph traversal algorithms

WHAT ARE GRAPHS

A **graph data structure** consists of a finite (and possibly mutable) set of vertices or nodes or points, together with a set of unordered pairs of these vertices for an undirected **graph** or a set of ordered pairs for a directed **graph**.

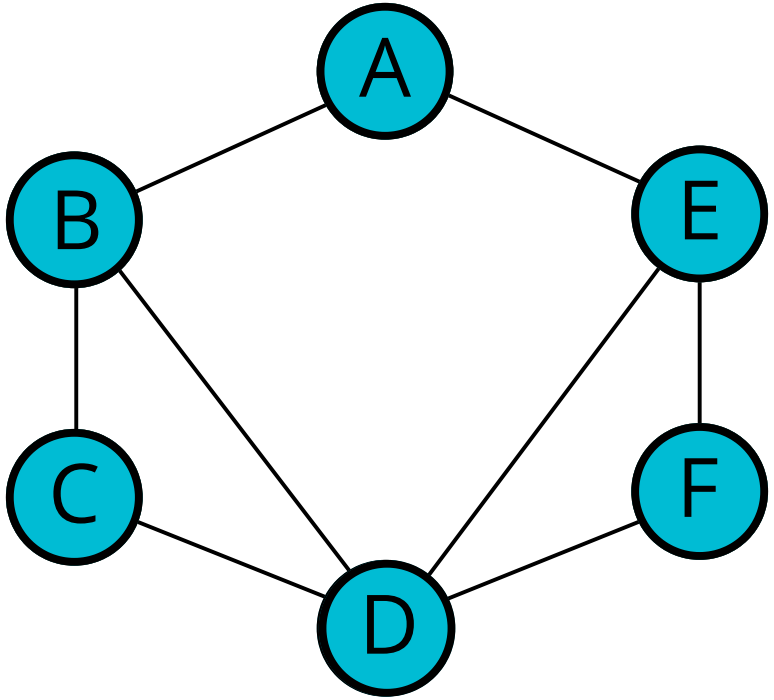
WHAT

Nodes

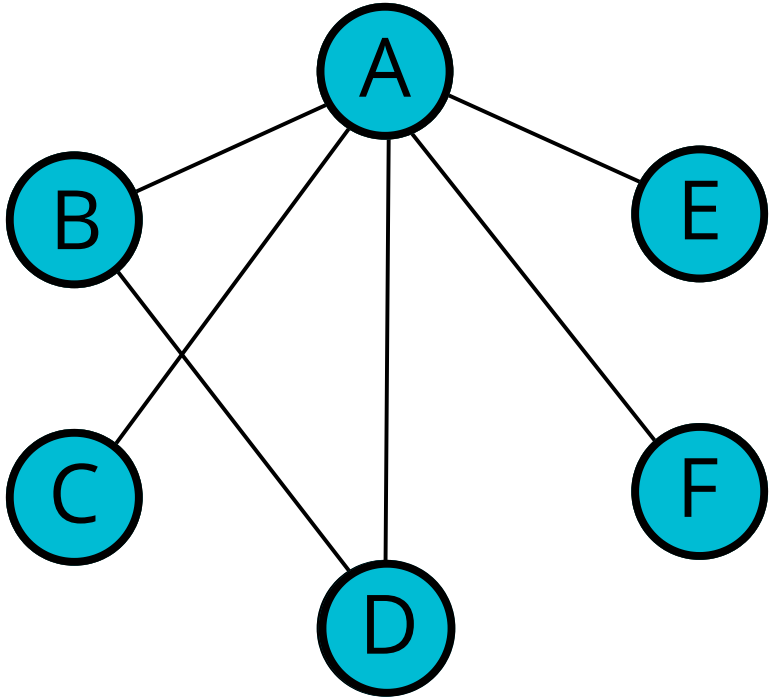
+

Connections

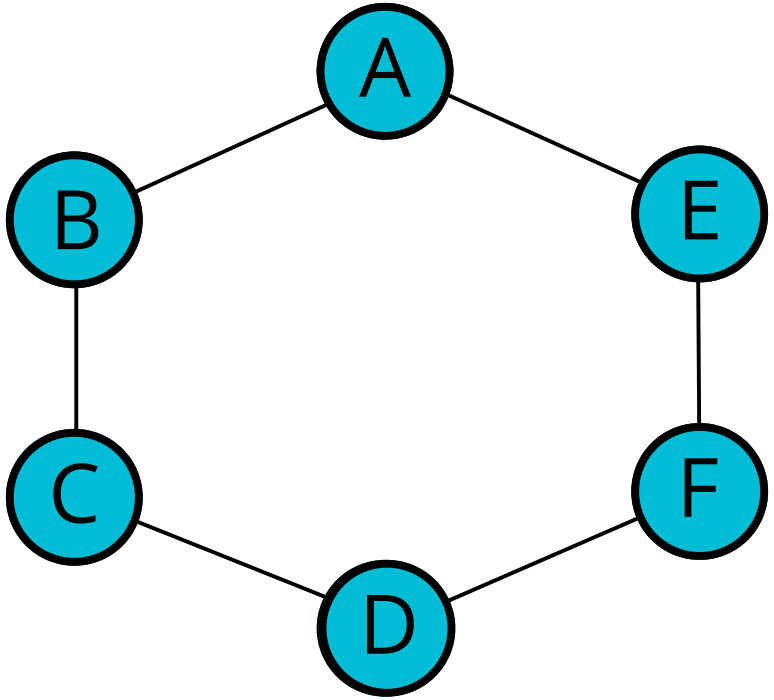
NODES + CONNECTIONS



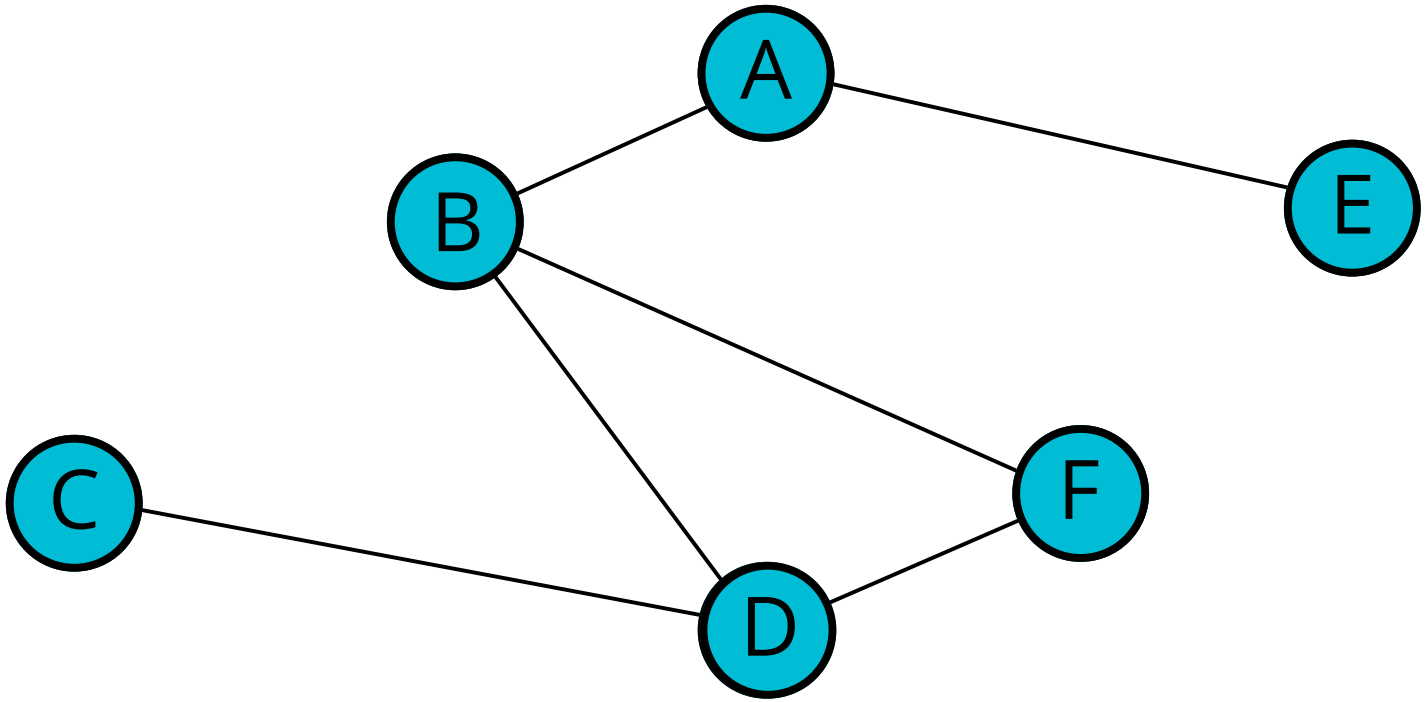
NODES + CONNECTIONS



NODES + CONNECTIONS

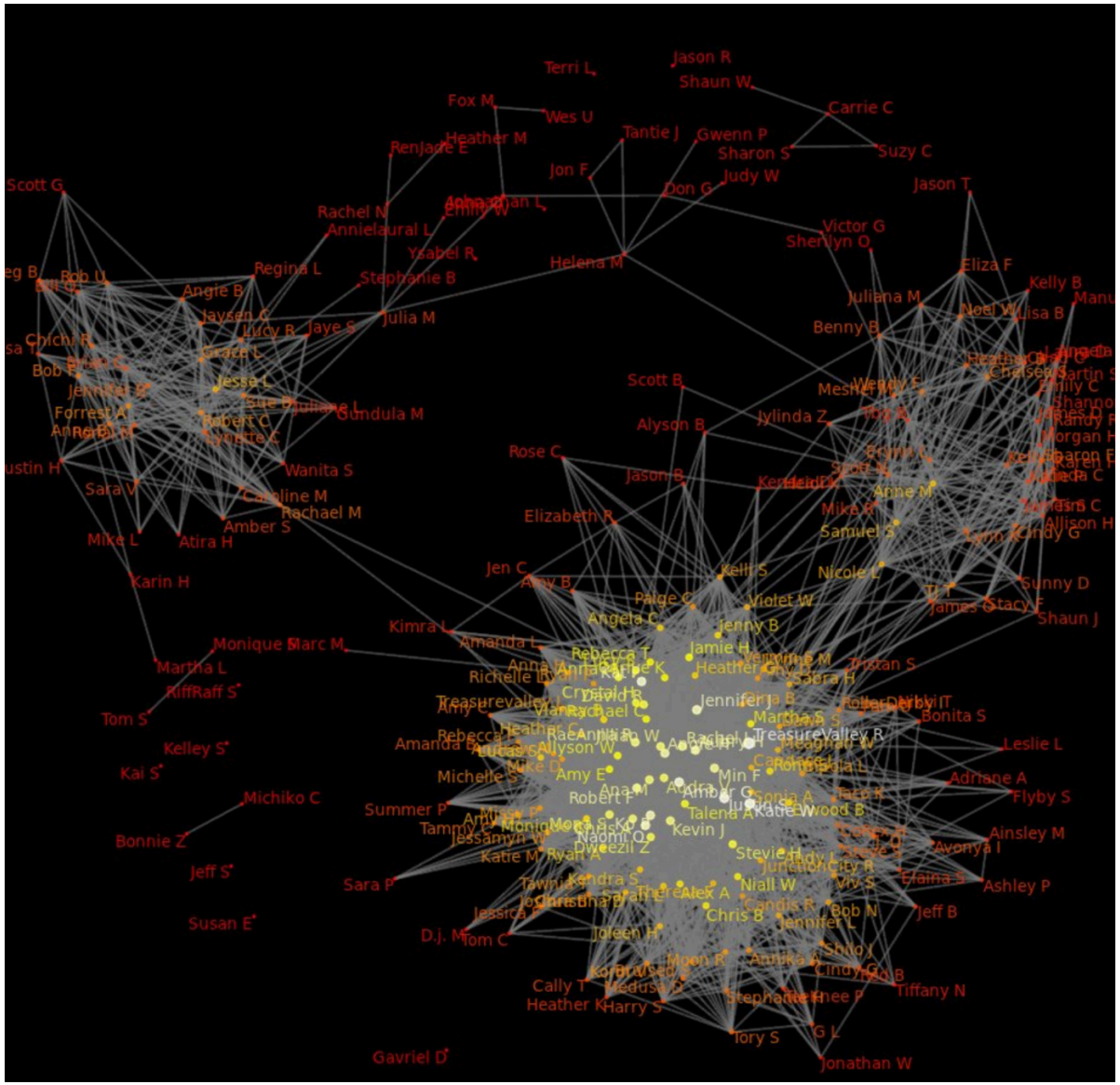


NODES + CONNECTIONS

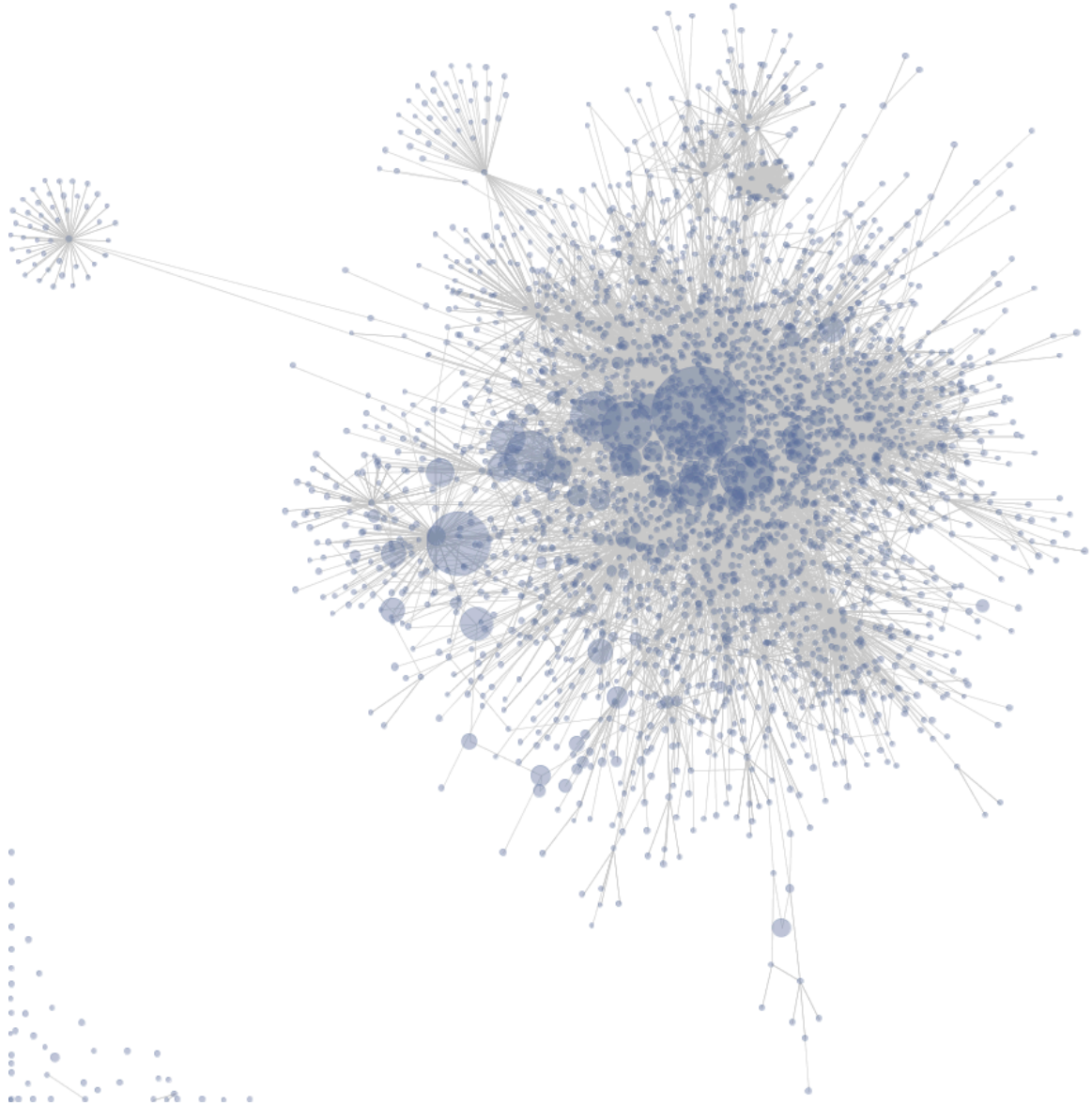


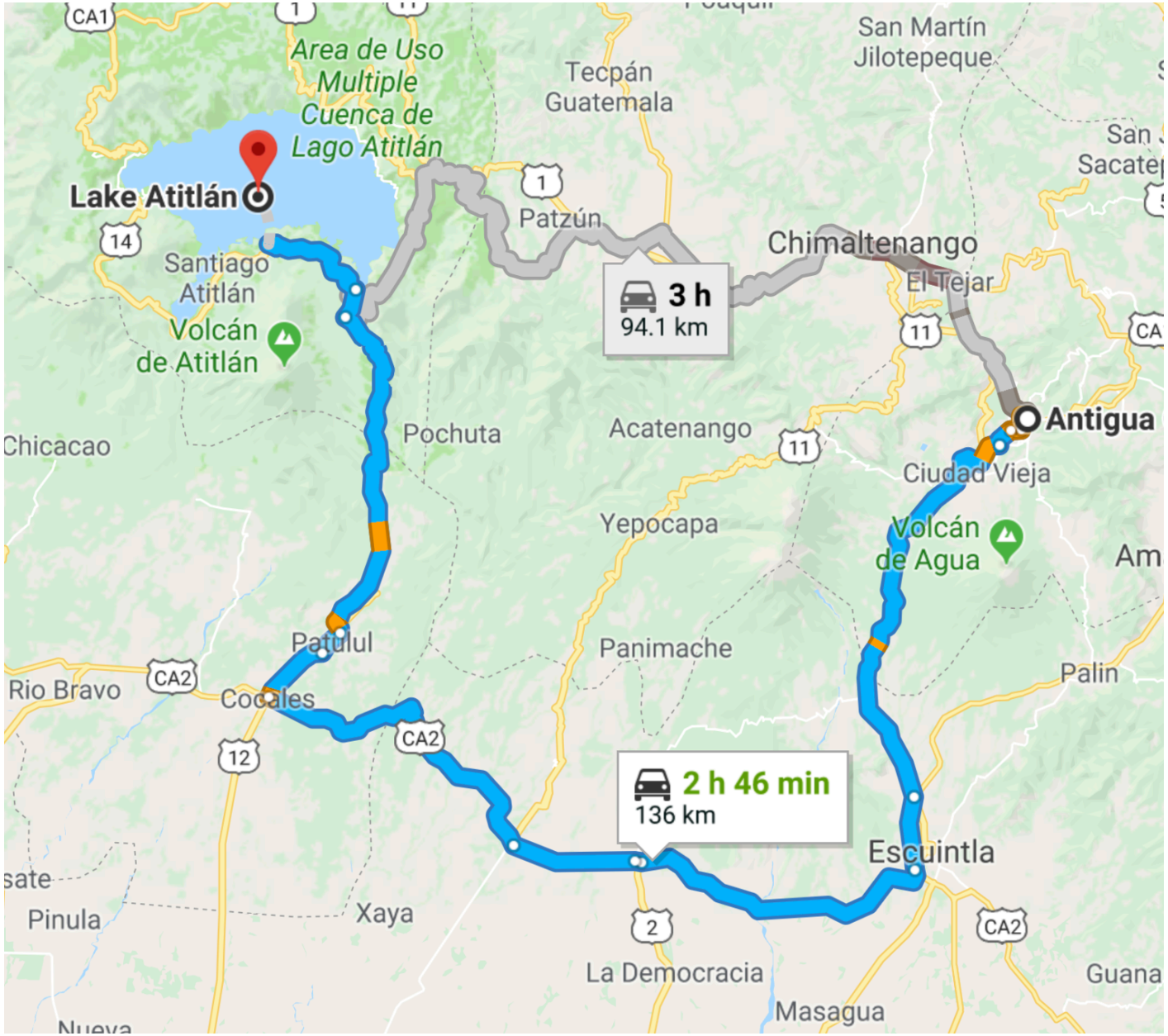
USES FOR GRAPHS

- Social Networks
- Location / Mapping
- Routing Algorithms
- Visual Hierarchy
- File System Optimizations
- **EVERYWHERE!**



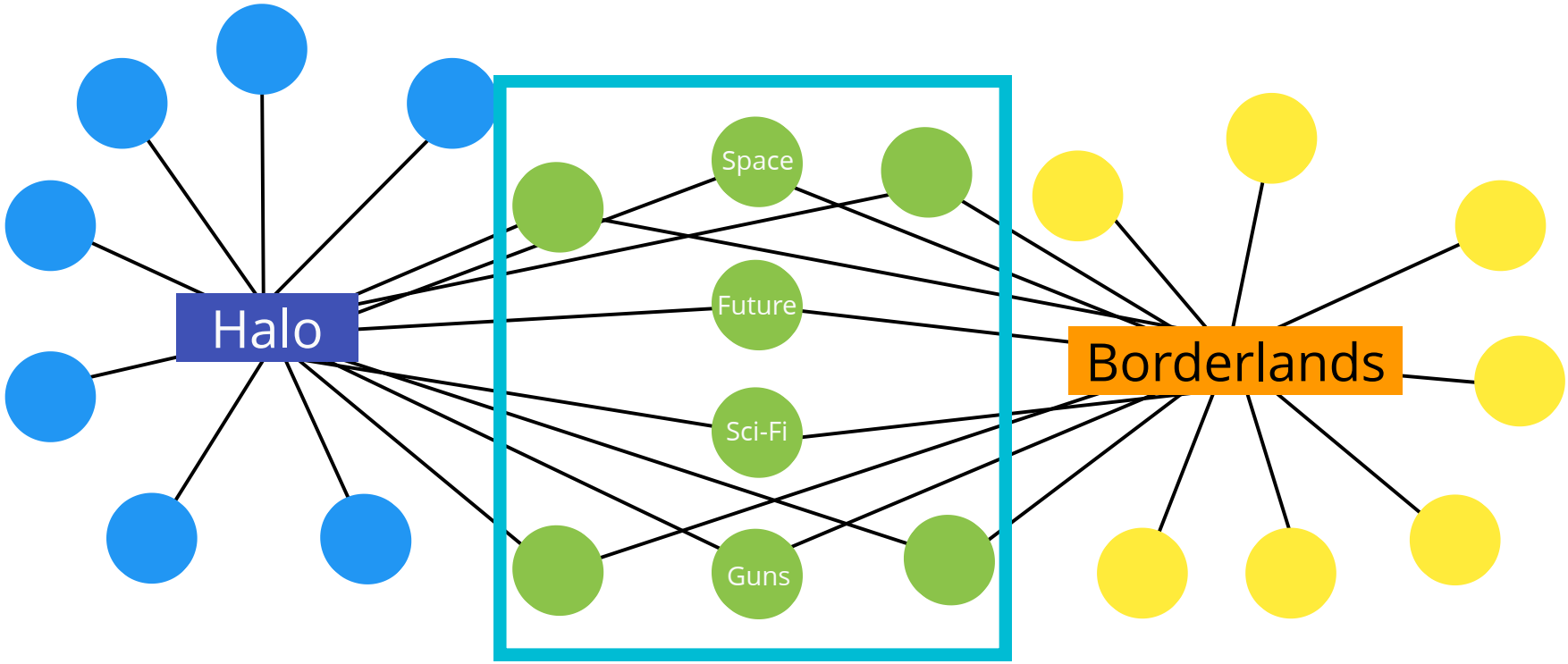
https://www.flickr.com/photos/kencf0618/6602925613/




Lake Atitlán
Area de Uso Multiple Cuenca de Lago Atitlán
Santiago Atitlán
Volcán de Atitlán
Tecpán Guatemala
Patzún
3 h
94.1 km
Chimaltenango
El Tejar
San Martín Jilotepeque
Antigua
Ciudad Vieja
Volcán de Agua
Acatenango
Yepocapa
Pochuta
Chicacao
Patulul
Cocales
Rio Bravo
Panimache
Palin
2 h 46 min
136 km
Escuintla
Xaya
Pinula
La Democracia
Masagua
CA1
CA2
14
12
2
11

Recommendations

- "People also watched"
- "You might also like..."
- "People you might know"
- "Frequently bought with"



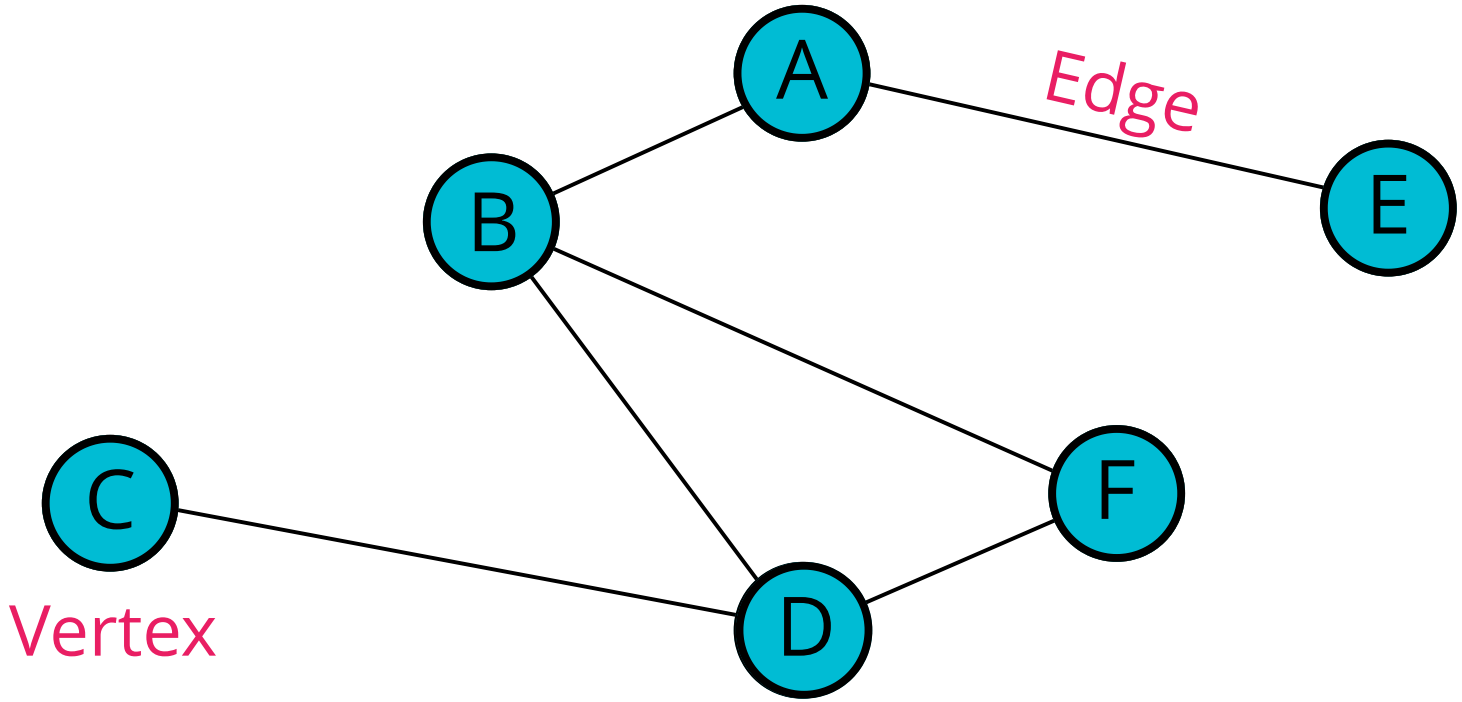
Things in Common

TYPES OF GRAPHS

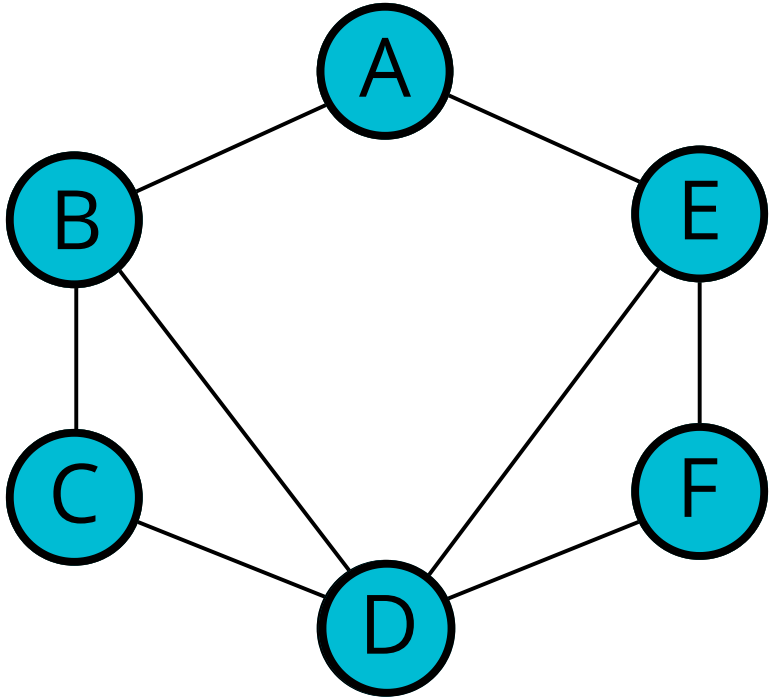
ESSENTIAL GRAPH TERMS

- **Vertex -** a node
- **Edge -** connection between nodes
- **Weighted/Unweighted -** values assigned to distances between vertices
- **Directed/Undirected -** directions assigned to distanced between vertices

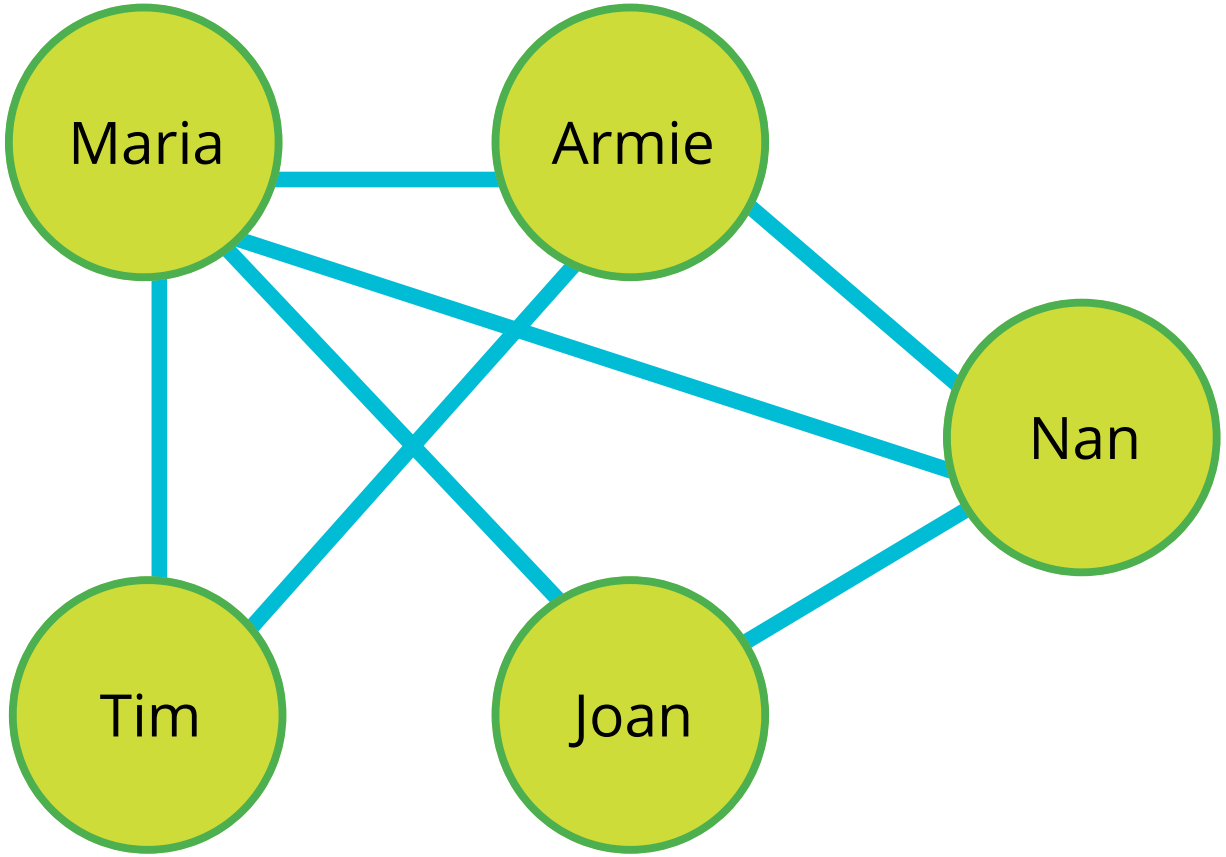
TERMINOLOGY



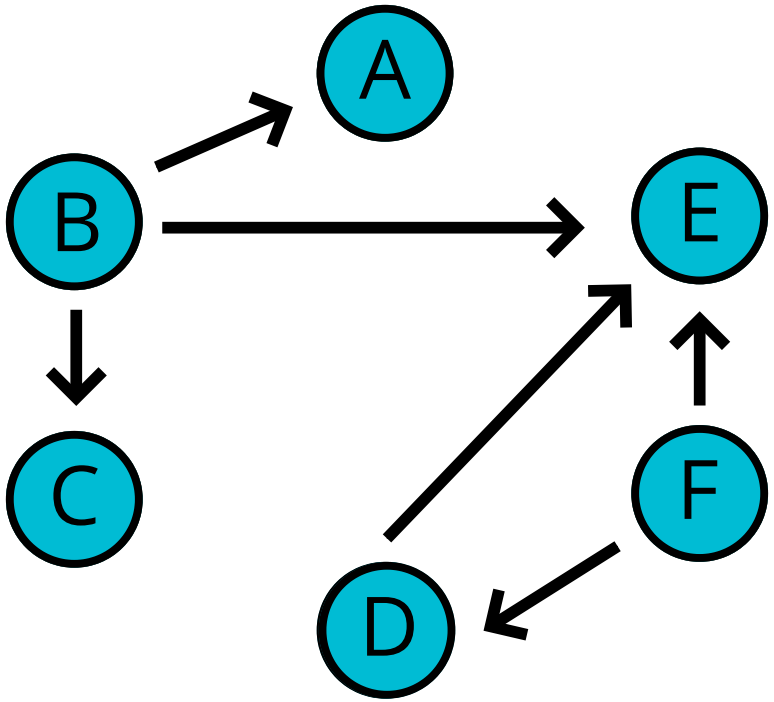
UNDIRECTED GRAPH



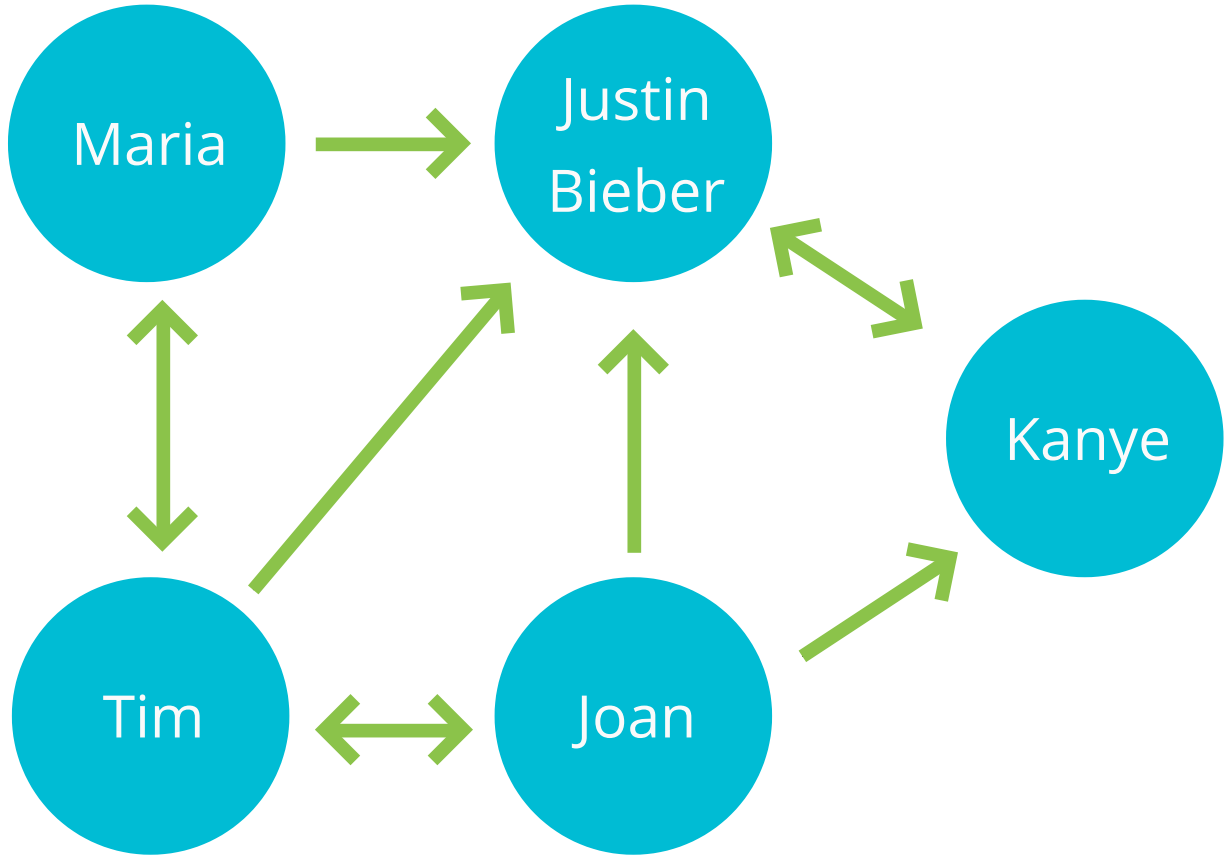
Facebook Friends Graph



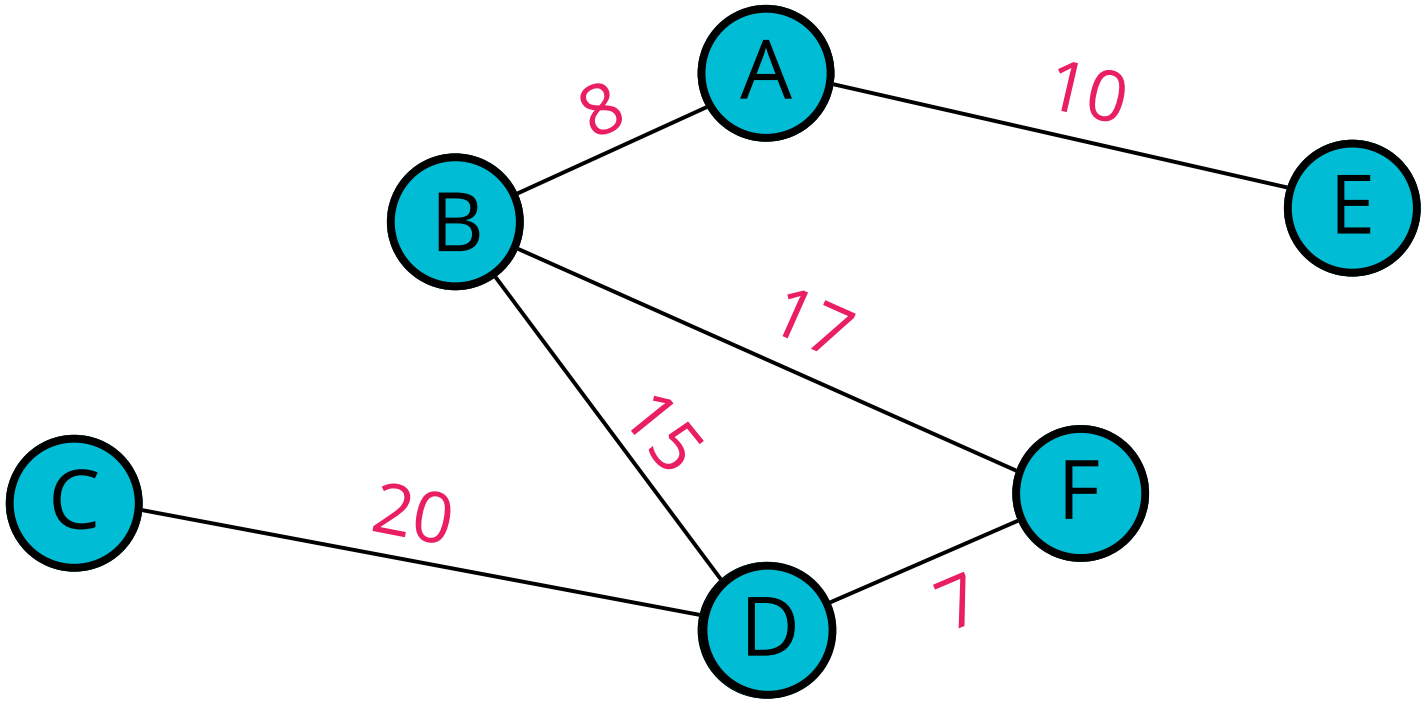
DIRECTED GRAPH

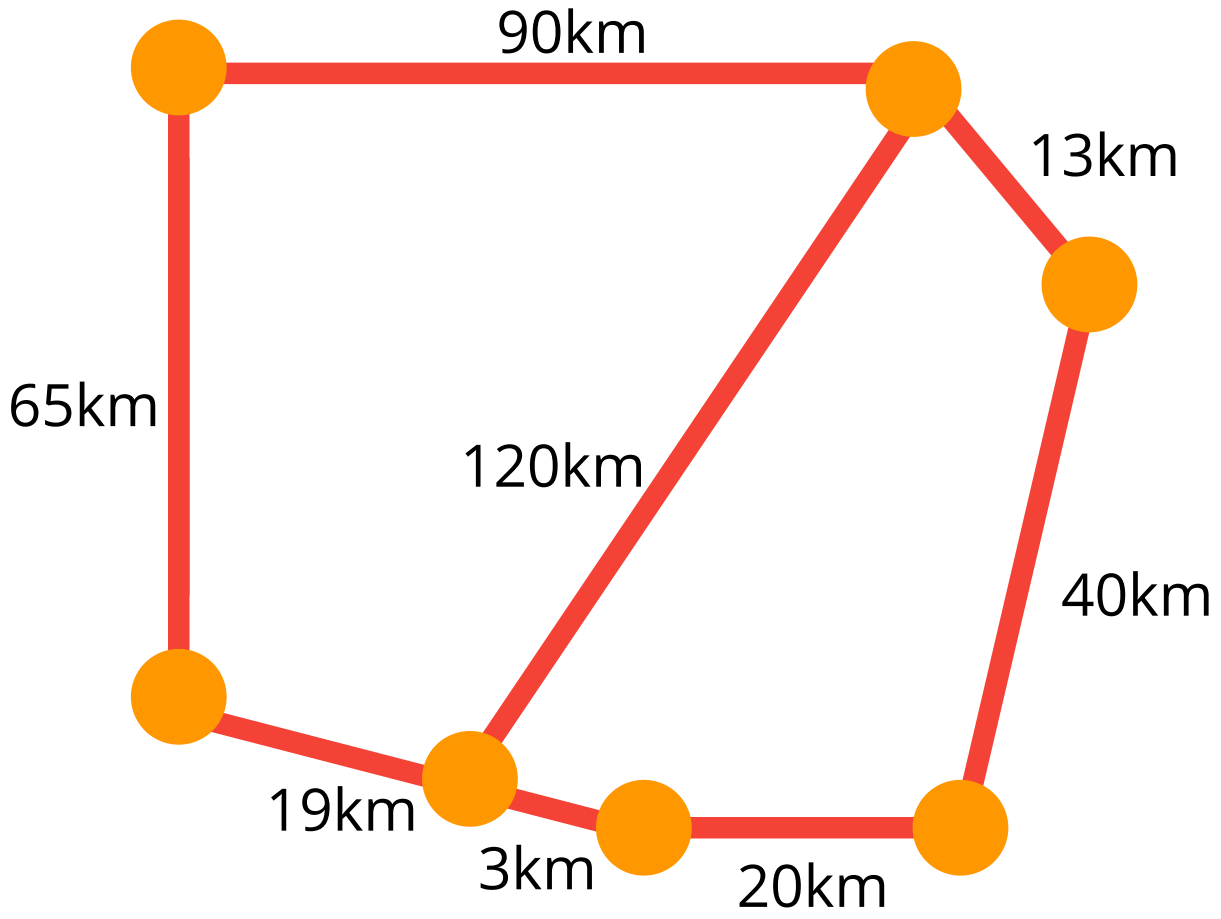


Instagram Followers Graph



WEIGHTED GRAPH

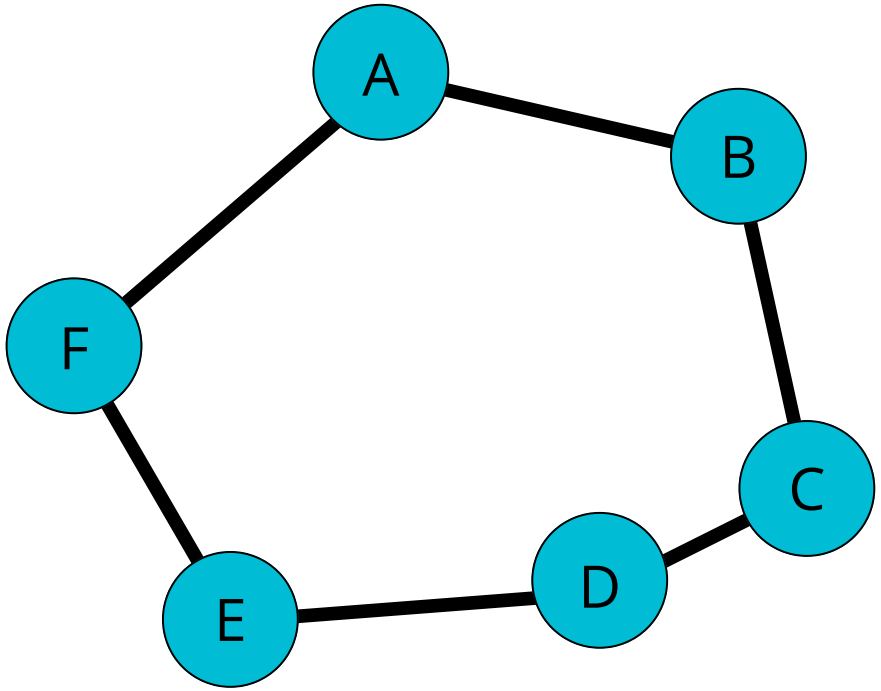



90km
13km
65km
120km
40km
19km
3km
20km

REPRESENTING A GRAPH

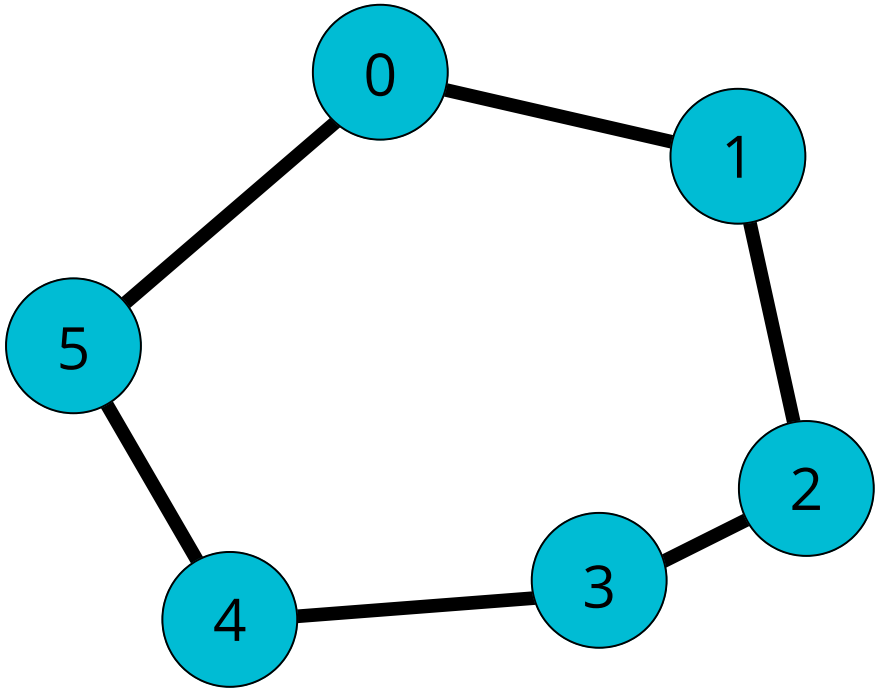
HOW DO WE STORE THIS???

ADJACENCY MATRIX



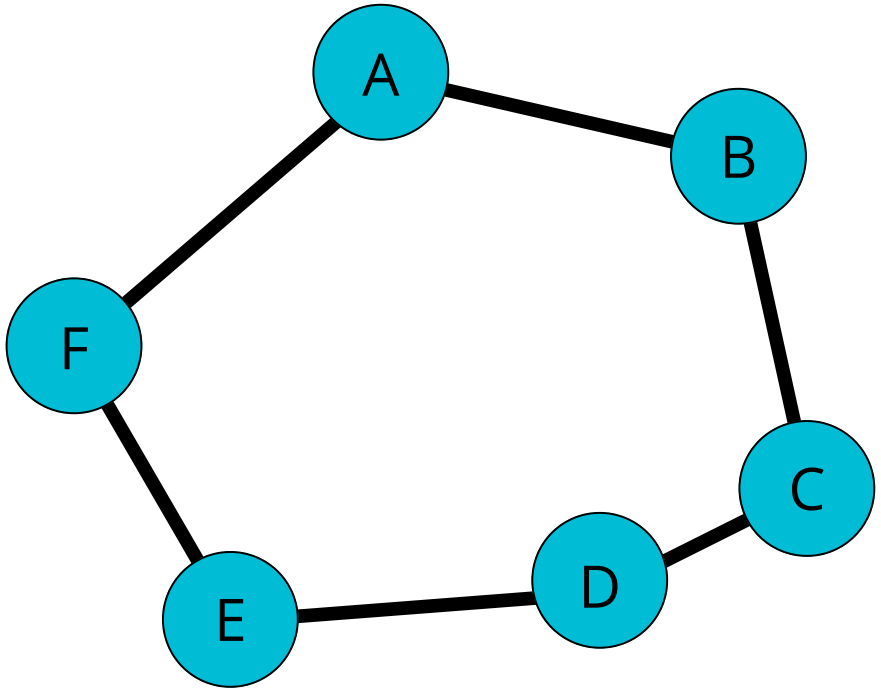
-	A	B	C	D	E	F
A	0	**1**	0	0	0	**1**
B	**1**	0	**1**	0	0	0
C	0	**1**	0	**1**	0	0
D	0	0	**1**	0	**1**	0
E	0	0	0	**1**	0	**1**
F	**1**	0	0	0	**1**	0

ADJACENCY LIST




```
   [
0    [1,5],
1    [0,2],
2    [1,3],
3    [2,4],
4    [3,5],
5    [4,0]
   ]
```

ADJACENCY LIST




```
{
    A: ["B", "F"],
    B: ["A", "C"],
    C: ["B", "D"],
    D: ["C", "E"],
    E: ["D", "F"],
    F: ["E", "A"]
}
```

DIFFERENCES & BIG O

|V| - number of vertices

|E| - number of edges

OPERATION	**ADJACENCY LIST**	**ADJACENCY MATRIX**
Add Vertex	O(1)	O(|V^2|)
Add Edge	O(1)	O(1)
Remove Vertex	O(|V| + |E|)	O(|V^2|)
Remove Edge	O(|E|)	O(1)
Query	O(|V| + |E|)	O(1)
Storage	O(|V| + |E|)	O(|V^2|)

Adjacency List

- Can take up less space (in sparse graphs)
- Faster to iterate over all edges
- Can be slower to lookup specific edge

vs.

Adjacency Matrix

- Takes up more space (in sparse graphs)
- Slower to iterate over all edges
- Faster to lookup specific edge

What will we use?

An Adjacency List

An Adjacency List
An Adjacency List

Why?

Most data in the real-world tends to lend itself to sparser and/or larger graphs

OUR GRAPH CLASS

```
class Graph {
    constructor(){
        this.adjacencyList = {}
    }
}
```

WE ARE BUILDING AN UNDIRECTED GRAPH

ADDING A VERTEX

- Write a method called addVertex, which accepts a name of a vertex
- It should add a key to the adjacency list with the name of the vertex and set its value to be an empty array

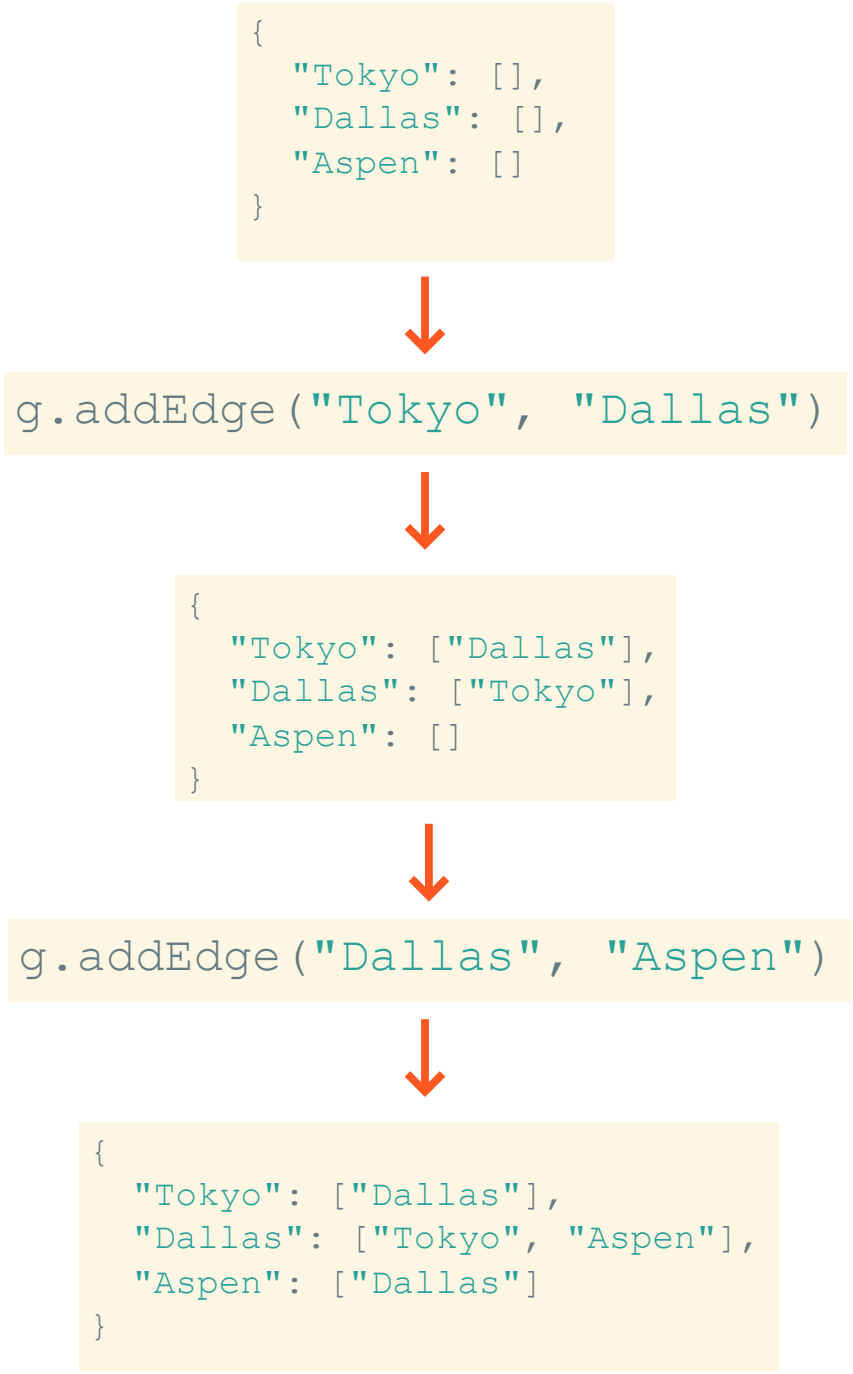
```
g.addVertex("Tokyo")
```

```
{
    "Tokyo": []
}
```

YOUR TURN

ADDING AN EDGE

- This function should accept two vertices, we can call them vertex1 and vertex2
- The function should find in the adjacency list the key of vertex1 and push vertex2 to the array
- The function should find in the adjacency list the key of vertex2 and push vertex1 to the array
- Don't worry about handling errors/invalid vertices


{
"Tokyo": [],
"Dallas": [],
"Aspen": []
}
g.addEdge("Tokyo", "Dallas")
{
"Tokyo": ["Dallas"],
"Dallas": ["Tokyo"],
"Aspen": []
}
g.addEdge("Dallas", "Aspen")
{
"Tokyo": ["Dallas"],
"Dallas": ["Tokyo", "Aspen"],
"Aspen": ["Dallas"]
}

YOUR TURN

REMOVING AN EDGE

- This function should accept two vertices, we'll call them vertex1 and vertex2
- The function should reassign the key of vertex1 to be an array that does not contain vertex2
- The function should reassign the key of vertex2 to be an array that does not contain vertex1
- Don't worry about handling errors/invalid vertices

REMOVING AN EDGE

```
{
  "Tokyo": ["Dallas"],
  "Dallas": ["Tokyo", "Aspen"],
  "Aspen": ["Dallas"]
}
```

```
g.removeEdge("Tokyo", "Dallas")
```

```
{
  "Tokyo": [],
  "Dallas": ["Aspen"],
  "Aspen": ["Dallas"]
}
```

YOUR TURN

REMOVING A VERTEX

- The function should accept a vertex to remove
- The function should loop as long as there are any other vertices in the adjacency list for that vertex
- Inside of the loop, call our **removeEdge** function with the vertex we are removing and any values in the adjacency list for that vertex
- delete the key in the adjacency list for that vertex

REMOVING A VERTEX

```
{
  "Tokyo": ["Dallas", "Hong Kong"],
  "Dallas": ["Tokyo", "Aspen", "Hong Kong", "Los Angeles"],
  "Aspen": ["Dallas"],
  "Hong Kong": ["Tokyo", "Dallas", "Los Angeles"],
  "Los Angeles": ["Hong Kong", "Dallas"]
}
```

```
g.removeVertex("Hong Kong")
```

```
{
  "Tokyo": ["Dallas"],
  "Dallas": ["Tokyo", "Aspen","Los Angeles"],
  "Aspen": ["Dallas"],
  "Los Angeles": ["Dallas"]
}
```

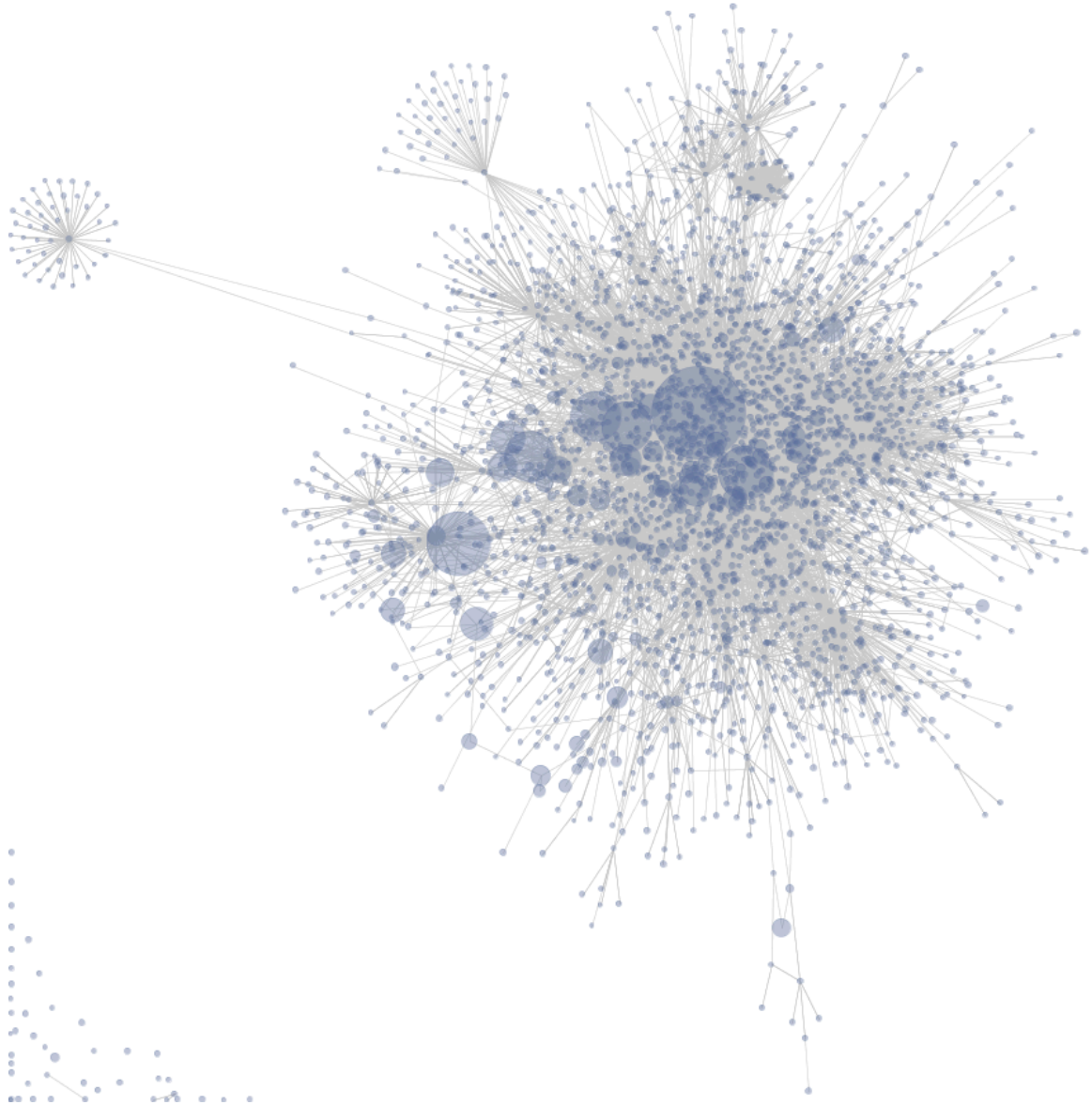
YOUR TURN

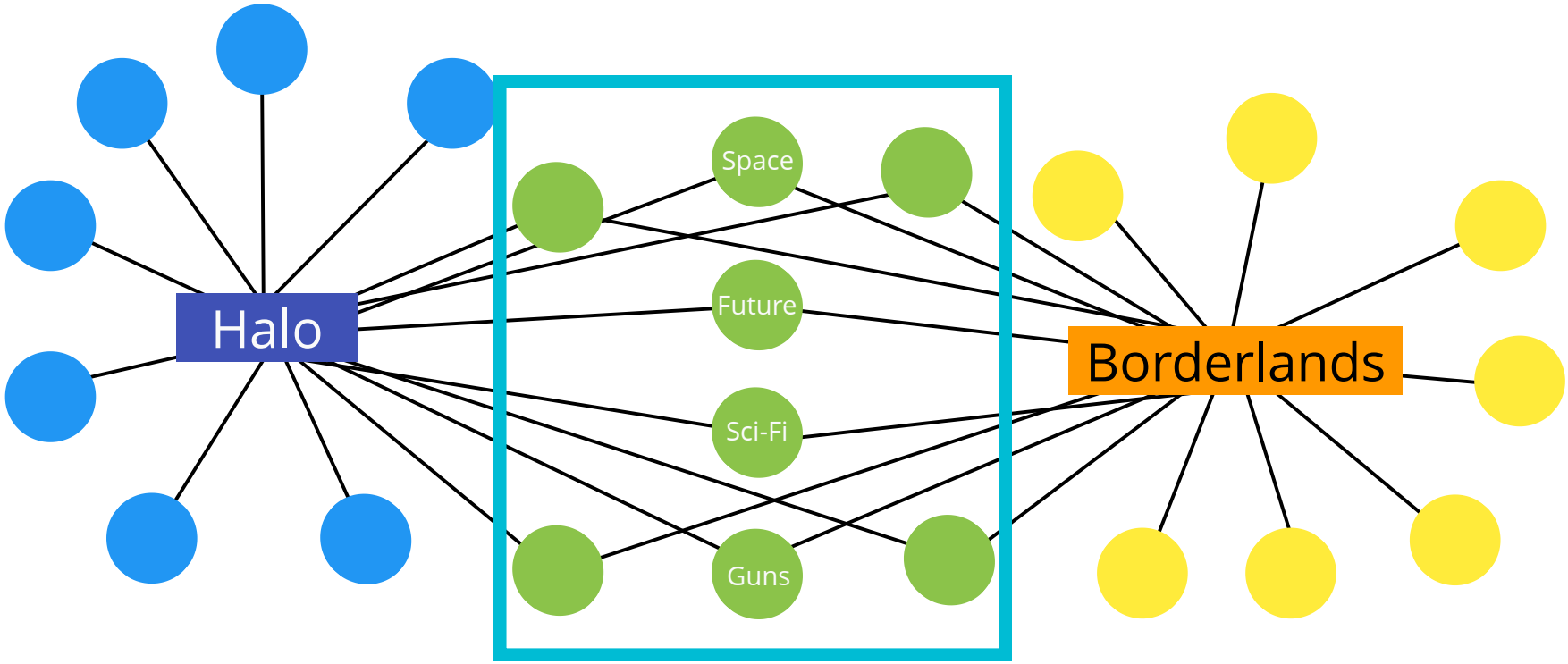
GRAPH TRAVERSAL

Visiting/Updating/Checking each vertex in a graph

GRAPH TRAVERSAL USES

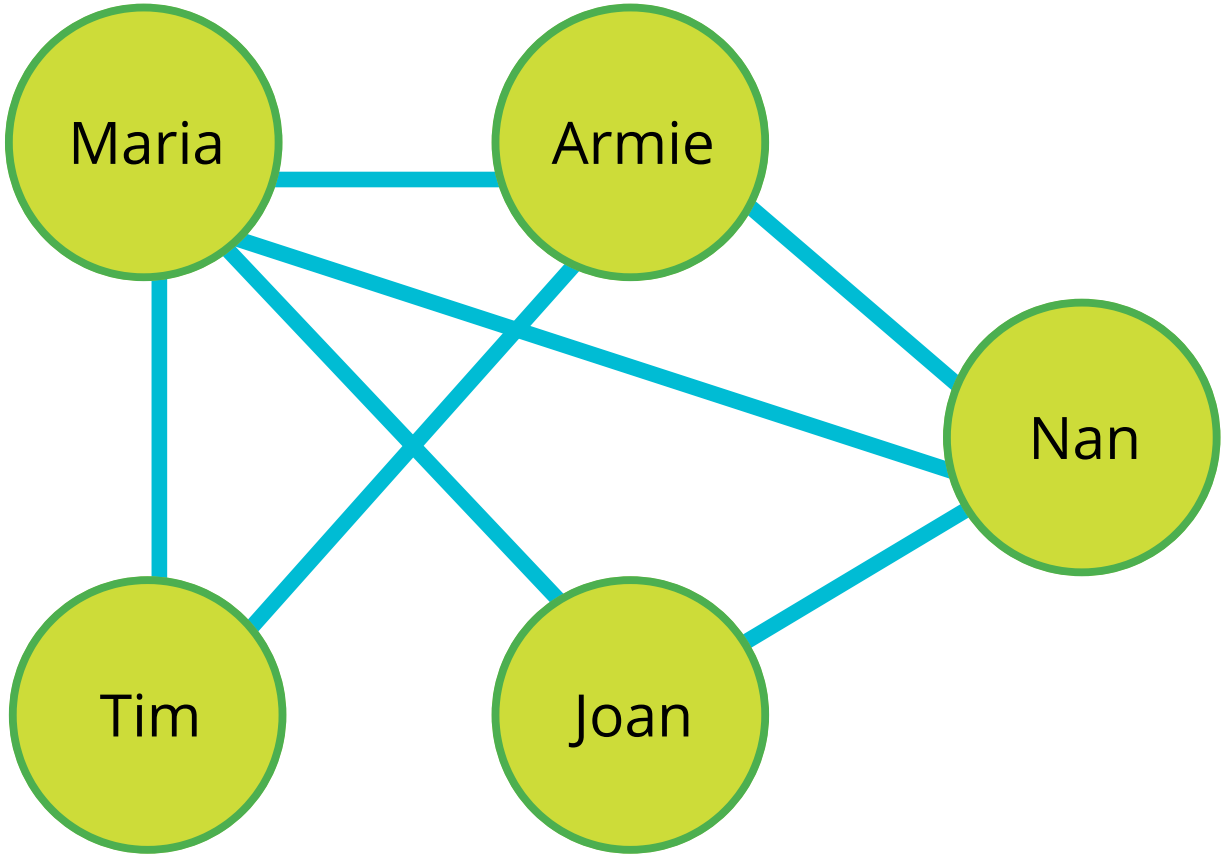
- Peer to peer networking
- Web crawlers
- Finding "closest" matches/recommendations
- Shortest path problems
  - GPS Navigation
  - Solving mazes
  - AI (shortest path to win the game)





Things in Common

Facebook Friends Graph

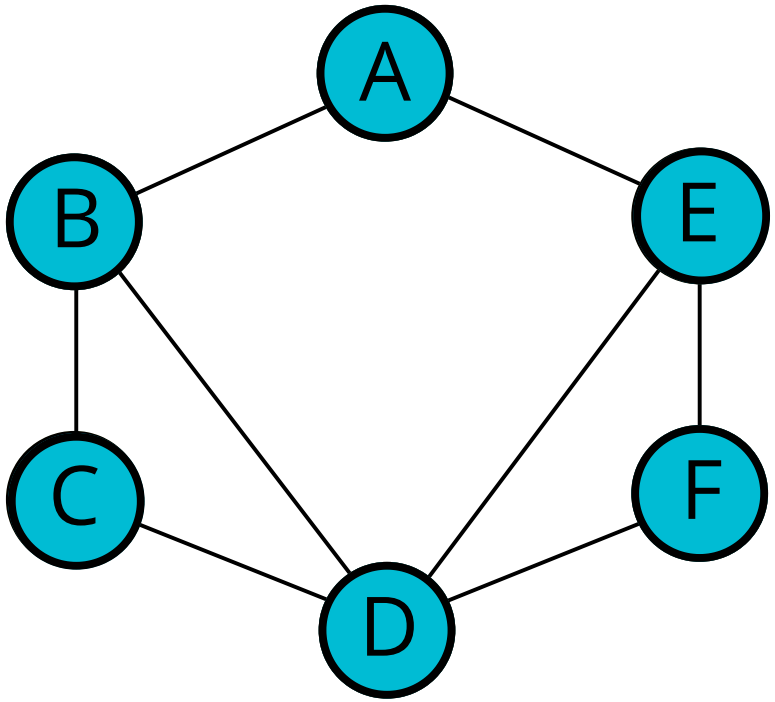


DEPTH FIRST

Explore as far as possible down one branch before "backtracking"

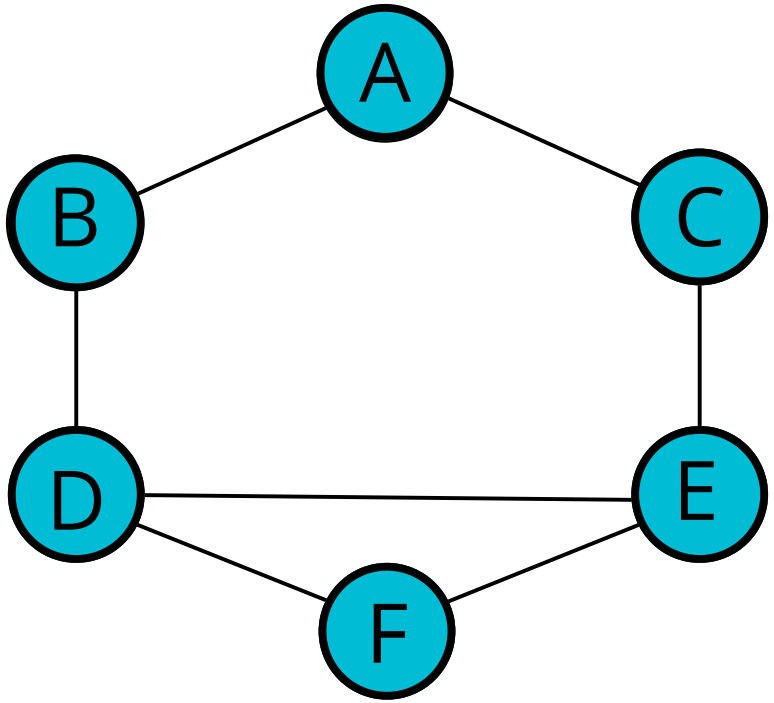
DEPTH FIRST TRAVERSAL

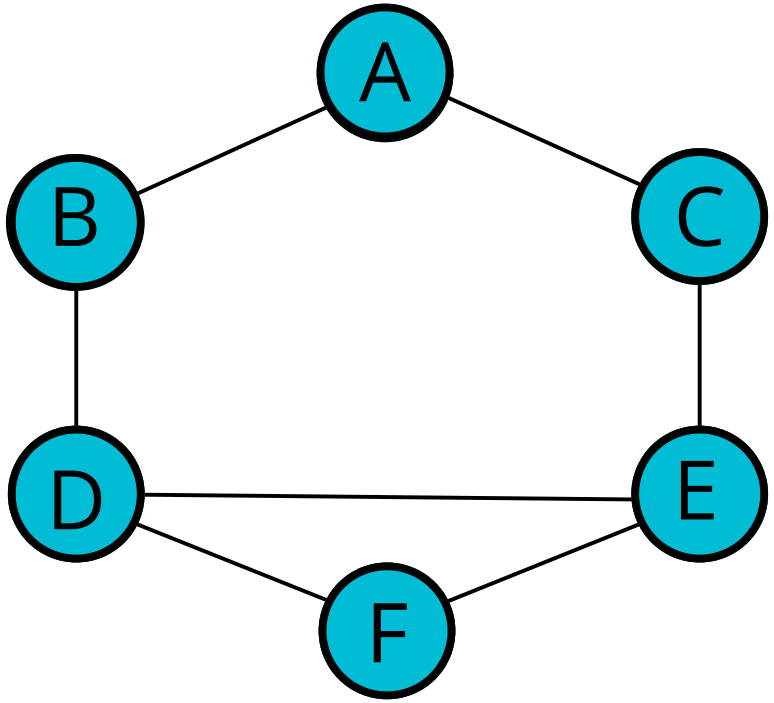
(STARTING FROM "A")



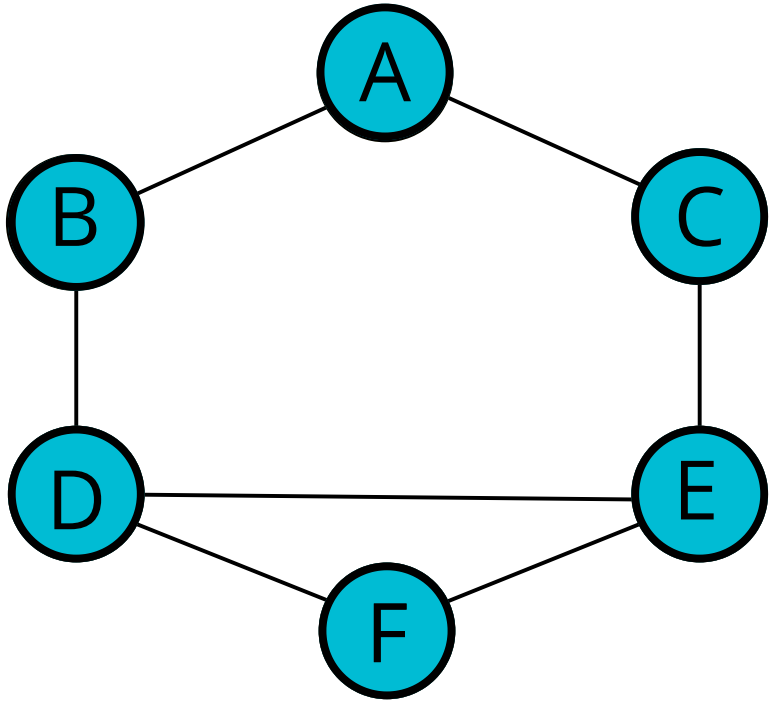
DEPTH FIRST TRAVERSAL

(STARTING FROM "A")






```
{
"A":["B", "C"],
"B":["A", "D"],
"C":["A", "E"],
"D":["B", "E", "F"],
"E":["C", "D", "F"],
"F":["D", "E"]
}
```




```
{
"A":["B", "C"],
"B":["A", "D"],
"C":["A", "E"],
"D":["B", "E", "F"],
"E":["C", "D", "F"],
"F":["D", "E"]
}
```

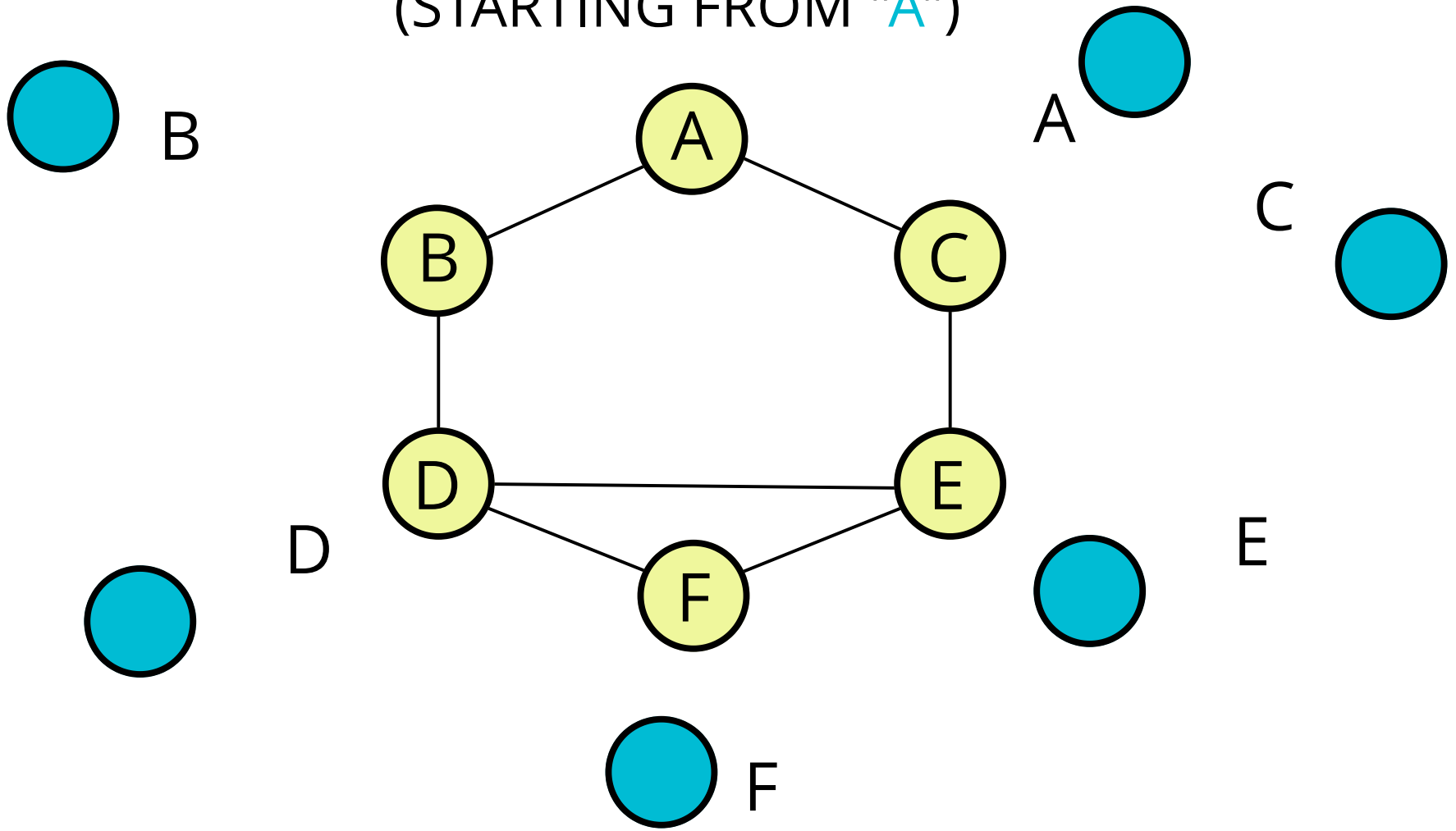
```
{
  "A": true,
  "B": true,
  "D": true,
  "E": true,
  "C": true,
  "F": true
}
```

```
g.addVertex("A")
g.addVertex("B")
g.addVertex("C")
g.addVertex("D")
g.addVertex("E")
g.addVertex("F")

g.addEdge("A","B")
g.addEdge("A","C")
g.addEdge("B","D")
g.addEdge("C","E")
g.addEdge("D","E")
g.addEdge("D","F")
g.addEdge("E","F")
```

DEPTH FIRST TRAVERSAL

(STARTING FROM "A")



DFS PSEUDOCODE

Recursive

```
DFS(vertex):
    if vertex is empty
        return (this is base case)
    add vertex to results list
    mark vertex as visited
    for each neighbor in vertex's neighbors:
        if neighbor is not visited:
            recursively call DFS on neighbor
```

VISITING THINGS

```
{
   "A": true,
   "B": true,
   "D": true
}
```

DEPTH FIRST TRAVERSAL

Recursive

- The function should accept a starting node
- Create a list to store the end result, to be returned at the very end
- Create an object to store visited vertices
- Create a helper function which accepts a vertex
  - The helper function should return early if the vertex is empty
  - The helper function should place the vertex it accepts into the visited object and push that vertex into the result array.
  - Loop over all of the values in the adjacencyList for that vertex
  - If any of those values have not been visited, recursively invoke the helper function with that vertex
- Invoke the helper function with the starting vertex
- Return the result array

DFS PSEUDOCODE

Iterative

```
DFS-iterative(start):
    let S be a stack
    S.push(start)
    while S is not empty
        vertex = S.pop()
        if vertex is not labeled as discovered:
            visit vertex (add to result list)
            label vertex as discovered
            for each of vertex's neighbors, N do
                S.push(N)
```

DEPTH FIRST TRAVERSAL

Iterative

- The function should accept a starting node
- Create a stack to help use keep track of vertices (use a list/array)
- Create a list to store the end result, to be returned at the very end
- Create an object to store visited vertices
- Add the starting vertex to the stack, and mark it visited
- While the stack has something in it:
  - Pop the next vertex from the stack
  - If that vertex hasn't been visited yet:
    - Mark it as visited
    - Add it to the result list
    - Push all of its neighbors into the stack
- Return the result array

DEPTH FIRST TRAVERSAL

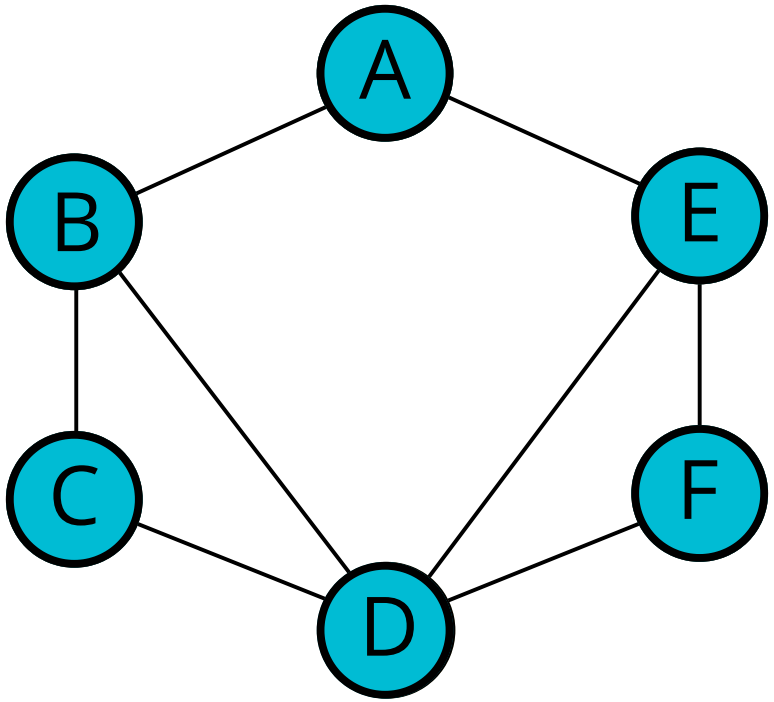
- The function should accept a starting node
- Create an object to store visited nodes and an array to store the result
- Create a helper function which accepts a vertex
- The helper function should place the vertex it accepts into the visited object and push that vertex into the results
- Loop over all of the values in the adjacencyList for that vertex
- If any of those values have not been visited, invoke the helper function with that vertex
- return the array of results

BREADTH FIRST

Visit neighbors at current depth first!

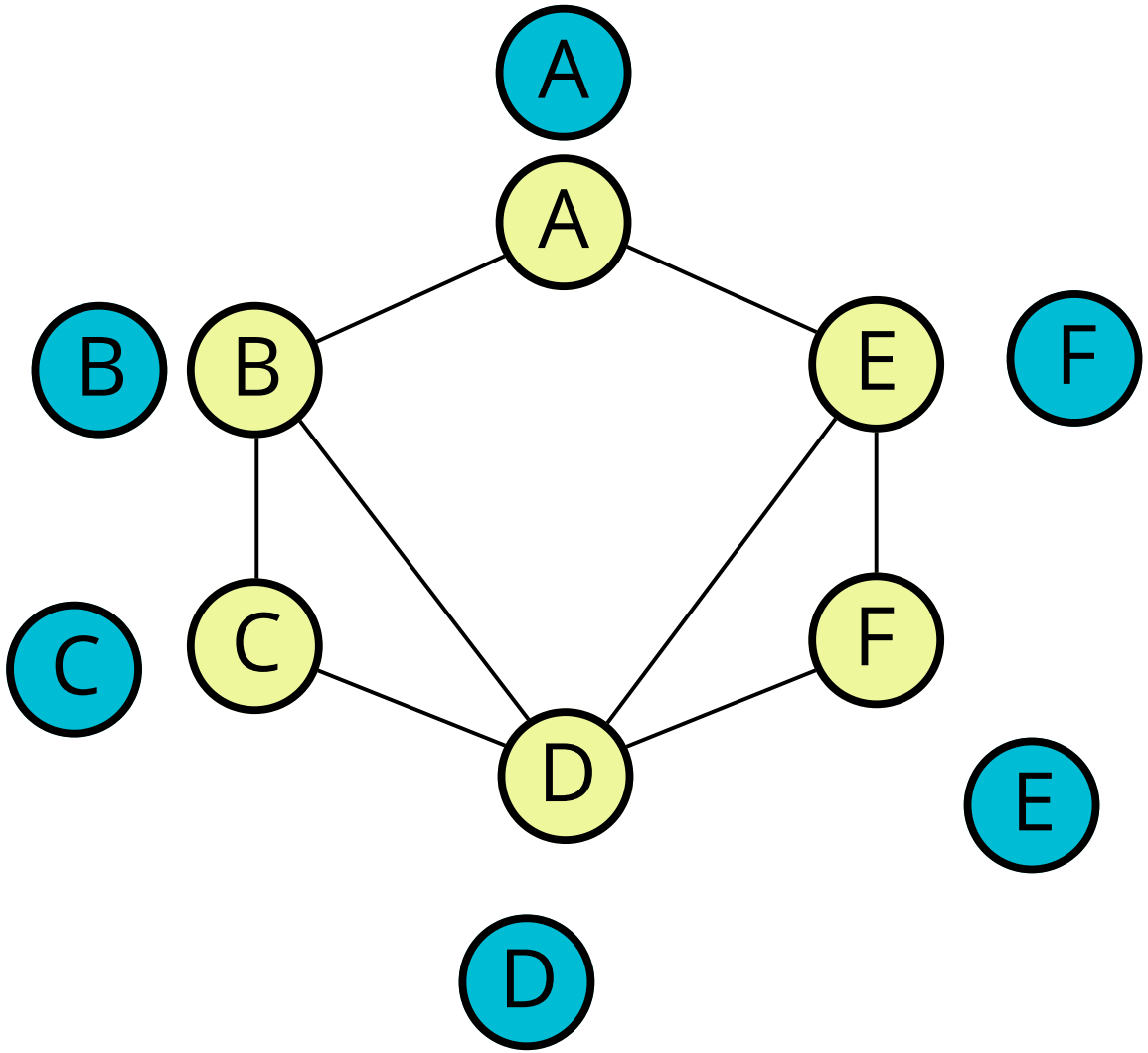
BREADTH FIRST TRAVERSAL

(STARTING FROM "A")



BREADTH FIRST

- This function should accept a starting vertex
- Create a queue (you can use an array) and place the starting vertex in it
- Create an array to store the nodes visited
- Create an object to store nodes visited
- Mark the starting vertex as visited
- Loop as long as there is anything in the queue
- Remove the first vertex from the queue and push it into the array that stores nodes visited
- Loop over each vertex in the adjacency list for the vertex you are visiting.
- If it is not inside the object that stores nodes visited, mark it as visited and enqueue that vertex
- Once you have finished looping, return the array of visited nodes


A
A
B
B
E
F
C
C
F
D
E
D

YOUR TURN

YOUR TURN

Shortest Path Algorithms

When working with weighted and directed/undirected graphs, we very commonly want to know how to get from one vertex to another! Better yet, how to do it **quickly.**

What's the fastest way to get from point A to point B?

Dijkstra's Algorithm

OBJECTIVES

- Understand the importance of Dijkstra's
- Implement a Weighted Graph
- Walk through the steps of Dijkstra's
- Implement Dijkstra's using a naive priority queue
- Implement Dijkstra's using a binary heap priority queue

WHAT IS IT

One of the most famous and widely used algorithms around!

Finds the shortest path between two vertices on a graph

"What's the fastest way to get from point A to point B?"

WHO WAS HE?

Edsger Dijkstra was a Dutch programmer, physicist, essayist, and all around smarty-pants

He helped to advance the field of computer science from an "art" to an academic discipline

Many of his discoveries and algorithms are still commonly used to this day.

HE DID A LOT...

Known for	Dijkstra's algorithm DJP algorithm First implementation of *ALGOL 60* Structured programming Semaphore THE multiprogramming system Multithreaded programming Concurrent programming Principles of distributed computing Mutual exclusion Call stack Fault-tolerant systems Self-stabilizing distributed systems Deadly embrace Shunting-yard algorithm Banker's algorithm Dining philosophers problem Predicate transformer semantics Guarded Command Language Weakest precondition calculus Smoothsort Separation of concerns Software architecture[1]

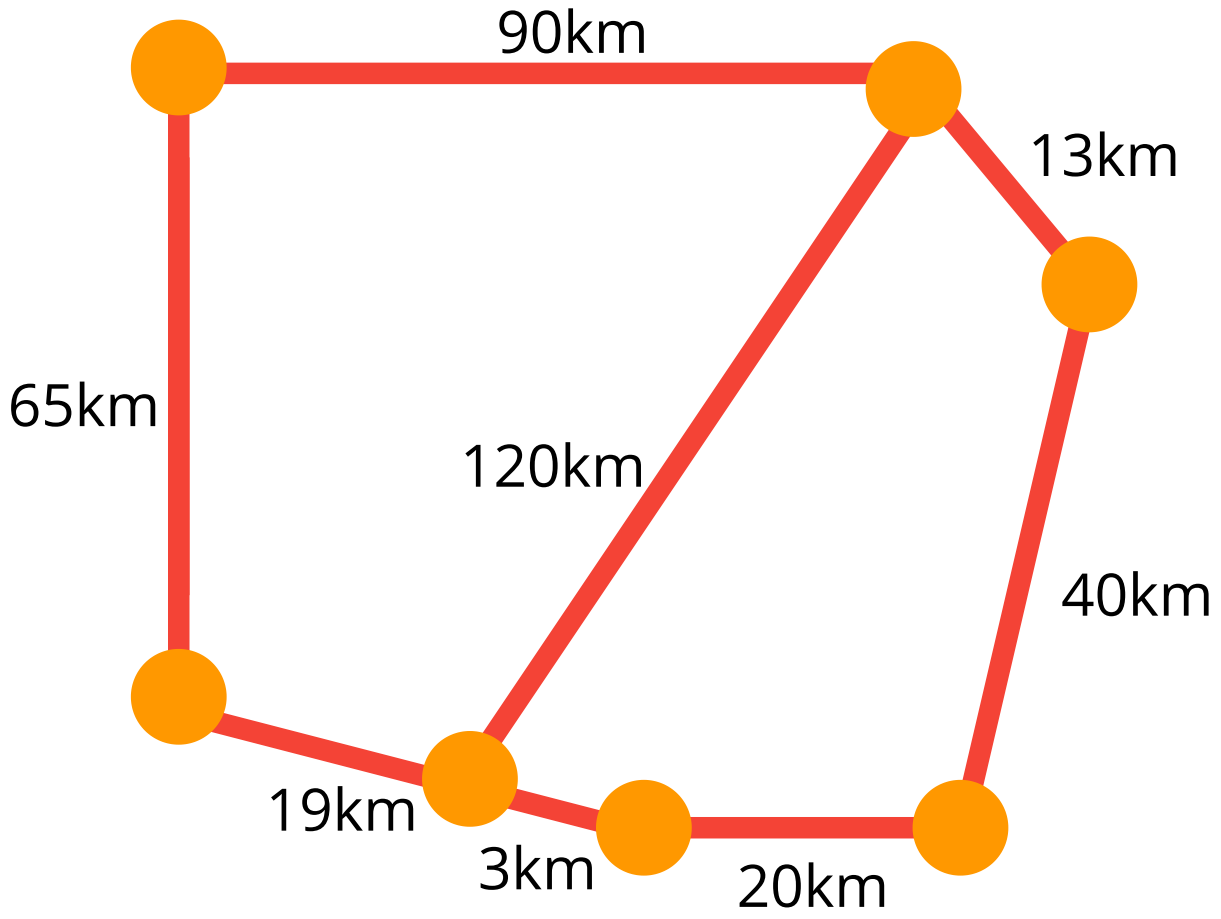
A QUOTE

What is the shortest way to travel from Rotterdam to Groningen, in general: from given city to given city. It is the algorithm for the shortest path, which I designed in about twenty minutes. One morning I was shopping in Amsterdam with my young fiancée, and tired, we sat down on the café terrace to drink a cup of coffee and I was just thinking about whether I could do this, and I then designed the algorithm for the shortest path. As I said, it was a twenty-minute invention. Eventually that algorithm became, to my great amazement, one of the cornerstones of my fame.

— Edsger Dijkstra, in an interview with Philip L. Frana, Communications of the ACM, 2001[2]

WHY IS IT USEFUL?

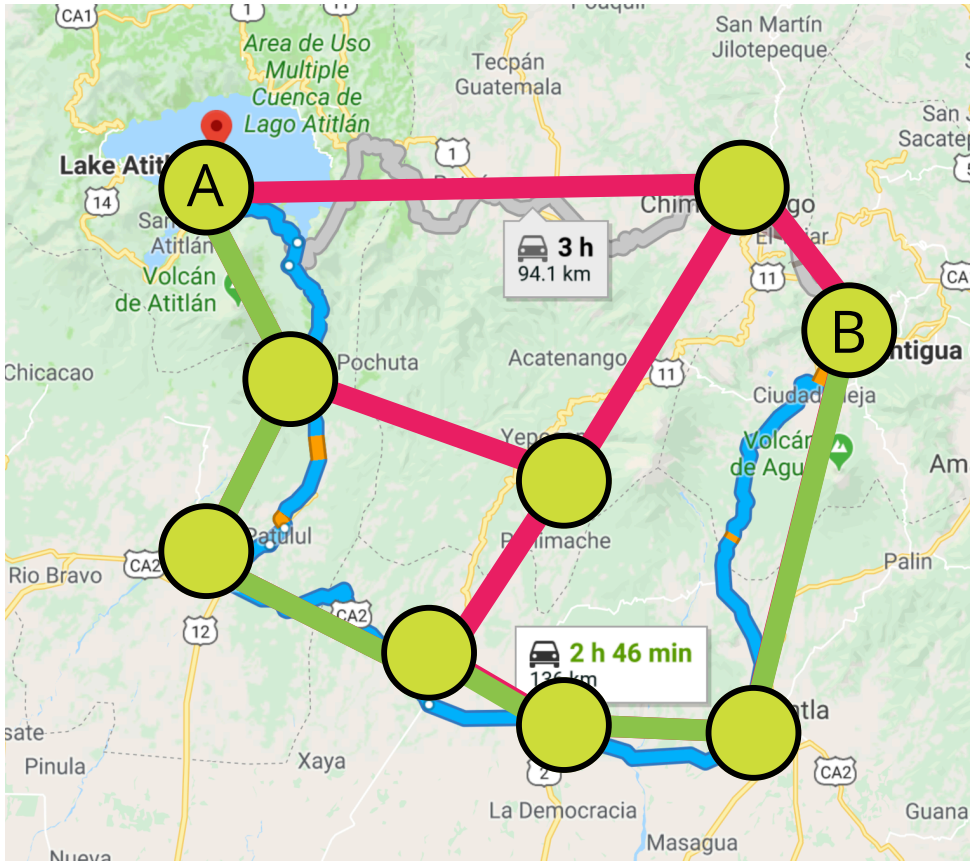
- GPS - finding fastest route
- Network Routing - finds open shortest path for data
- Biology - used to model the spread of viruses among humans
- Airline tickets - finding cheapest route to your destination
- Many other uses!


90km
13km
65km
120km
40km
19km
3km
20km

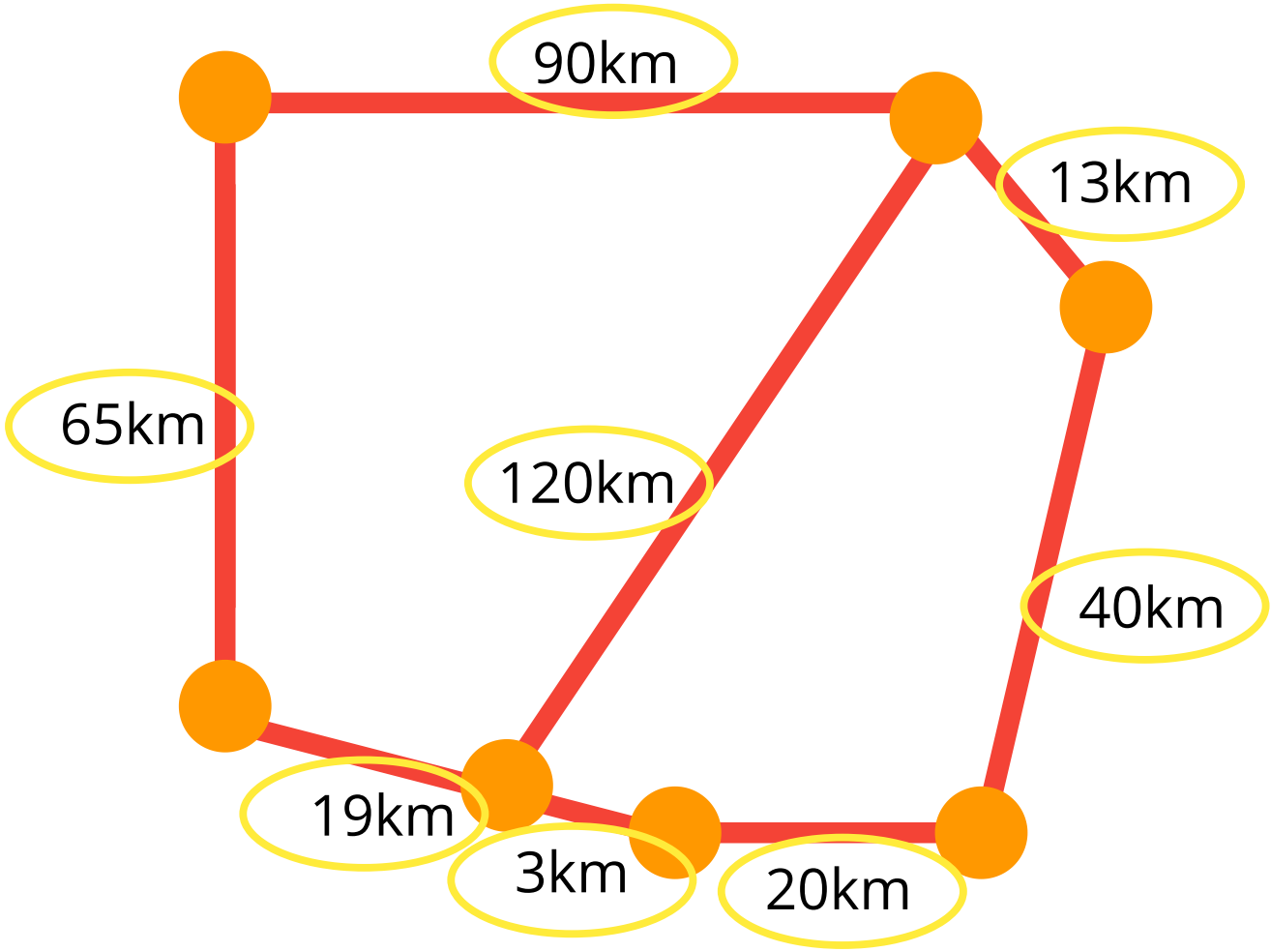
HOW DOES GPS WORK?

Find the shortest path from

Santiago to Antigua



OUR GRAPH CAN'T DO THIS YET!



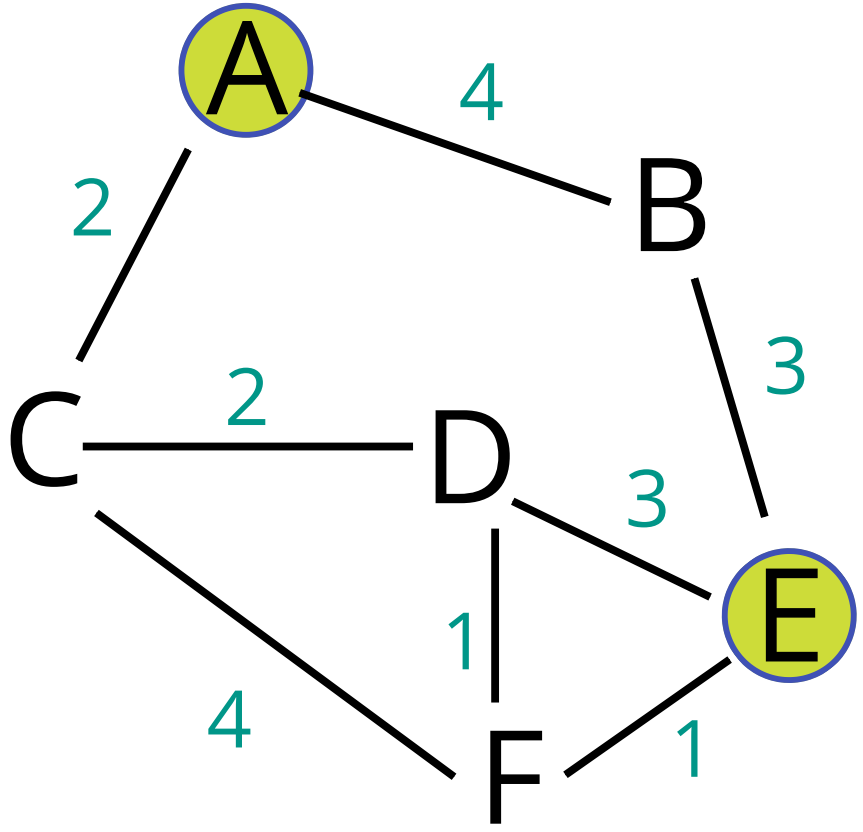
Let's Write A Weighted Graph

Now on to Dikstra's

Djikstra's

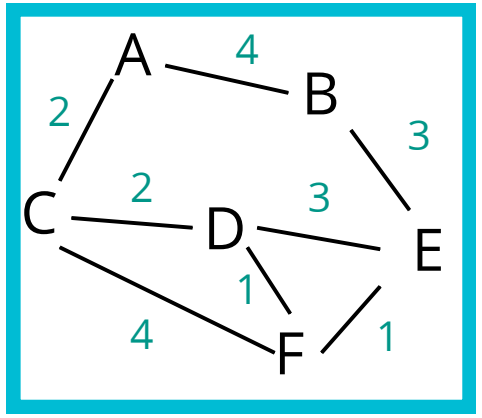
Dijkstra's

Find the shortest path from A to E



THE APPROACH

1. Every time we look to visit a new node, we pick the node with the smallest known distance to visit first.
2. Once we've moved to the node we're going to visit, we look at each of its neighbors
3. For each neighboring node, we calculate the distance by summing the total edges that lead to the node we're checking *from the starting node.*
4. If the new total distance to a node is less than the previous total, we store the new shorter distance for that node.



FIND THE SHORTEST PATH FROM A TO E

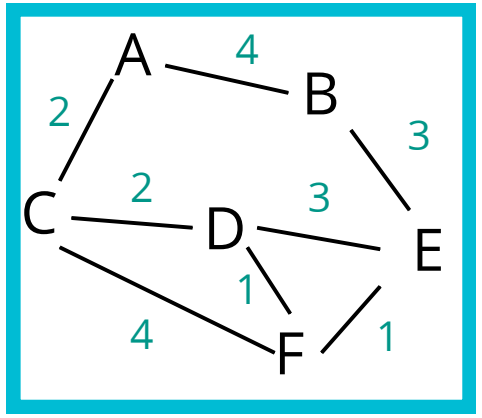
Vertex	Shortest Dist From A
A	
B	
C	
D	
E	
F	

Visited:

```
[]
```

Previous:

```
{
    A: null,
    B: null,
    C: null,
    D: null,
    E: null,
    F: null
}
```



FIND THE SHORTEST PATH FROM A TO E

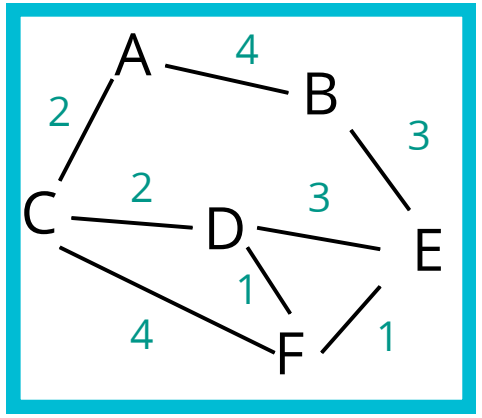
Vertex	Shortest Dist From A
A	0
B	Infinity
C	Infinity
D	Infinity
E	Infinity
F	Infinity

Visited:

```
[]
```

Previous:

```
{
    A: null,
    B: null,
    C: null,
    D: null,
    E: null,
    F: null
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...A

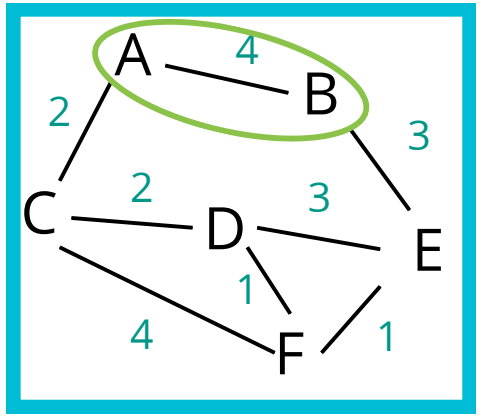
Vertex	Shortest Dist From A
A	0
B	Infinity
C	Infinity
D	Infinity
E	Infinity
F	Infinity

Visited:

```
[A]
```

Previous:

```
{
    A: null,
    B: null,
    C: null,
    D: null,
    E: null,
    F: null
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...A

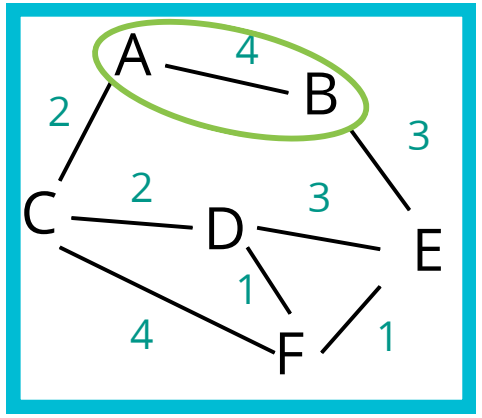
Vertex	Shortest Dist From A
A	0
B	Infinity
C	Infinity
D	Infinity
E	Infinity
F	Infinity

Visited:

```
[A]
```

Previous:

```
{
    A: null,
    B: null,
    C: null,
    D: null,
    E: null,
    F: null
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...A

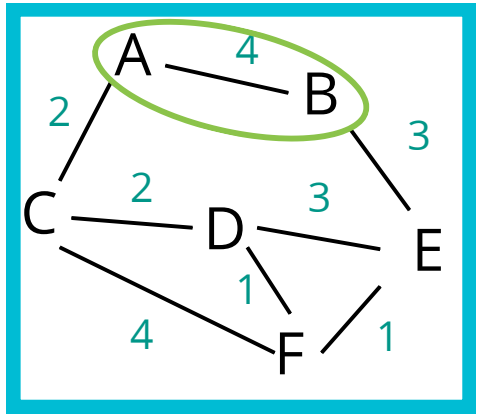
Vertex	Shortest Dist From A
A	0
B	~~Infinity~~,4
C	Infinity
D	Infinity
E	Infinity
F	Infinity

Visited:

```
[A]
```

Previous:

```
{
    A: null,
    B: null,
    C: null,
    D: null,
    E: null,
    F: null
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...A

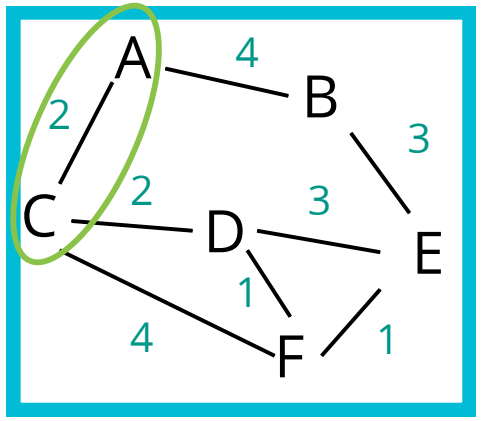
Vertex	Shortest Dist From A
A	0
B	~~Infinity~~,4
C	Infinity
D	Infinity
E	Infinity
F	Infinity

Visited:

```
[A]
```

Previous:

```
{
    A: null,
    B: A,
    C: null,
    D: null,
    E: null,
    F: null
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...A

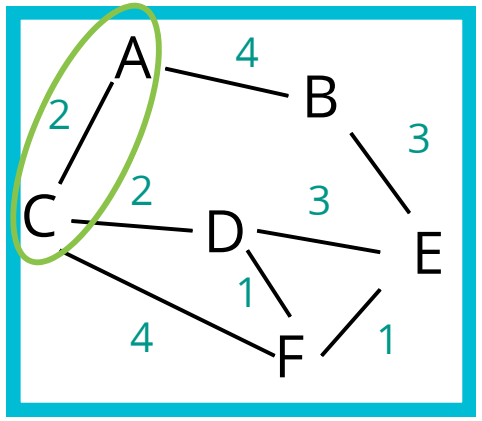
Vertex	Shortest Dist From A
A	0
B	~~Infinity~~,4
C	Infinity
D	Infinity
E	Infinity
F	Infinity

Visited:

```
[A]
```

Previous:

```
{
    A: null,
    B: A,
    C: null,
    D: null,
    E: null,
    F: null
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...A

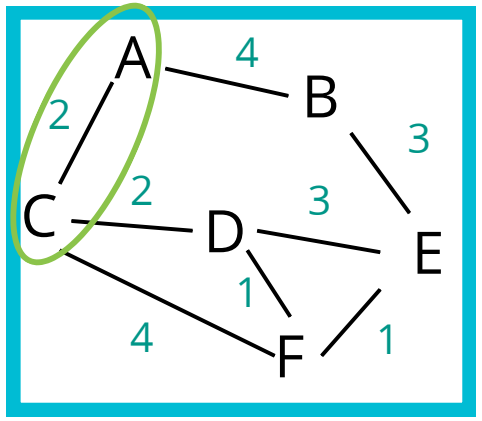
Vertex	Shortest Dist From A
A	0
B	~~Infinity~~,4
C	~~Infinity~~,2
D	Infinity
E	Infinity
F	Infinity

Visited:

```
[A]
```

Previous:

```
{
    A: null,
    B: A,
    C: null,
    D: null,
    E: null,
    F: null
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...A

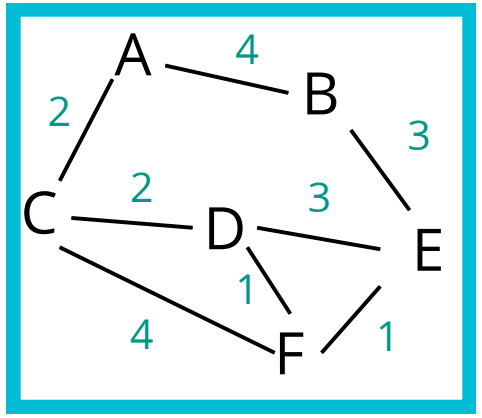
Vertex	Shortest Dist From A
A	0
B	~~Infinity~~,4
C	~~Infinity~~,2
D	Infinity
E	Infinity
F	Infinity

Visited:

```
[A]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: null,
    E: null,
    F: null
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...C

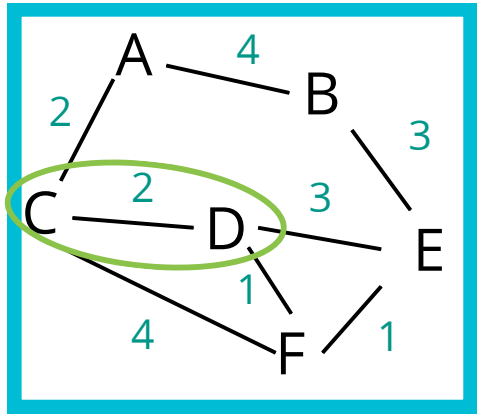
Vertex	Shortest Dist From A
~~A~~	0
B	~~Infinity~~,4
C	~~Infinity~~,2
D	Infinity
E	Infinity
F	Infinity

Visited:

```
[A]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: null,
    E: null,
    F: null
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...C

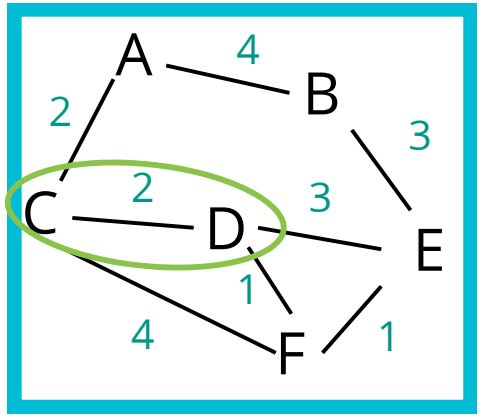
Vertex	Shortest Dist From A
~~A~~	0
B	~~Infinity~~,4
C	~~Infinity~~,2
D	Infinity
E	Infinity
F	Infinity

Visited:

```
[A,C]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: null,
    E: null,
    F: null
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...C

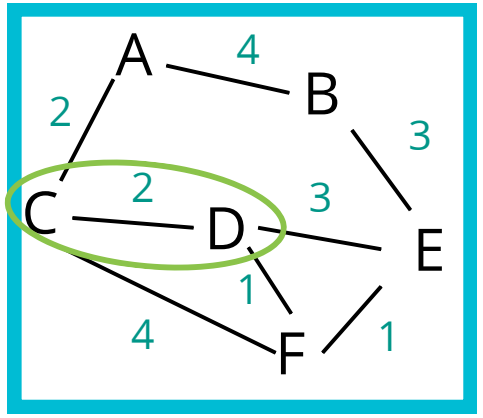
Vertex	Shortest Dist From A
~~A~~	0
B	~~Infinity~~,4
C	~~Infinity~~,2
D	~~Infinity~~,4
E	Infinity
F	Infinity

Visited:

```
[A,C]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: null,
    E: null,
    F: null
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...C

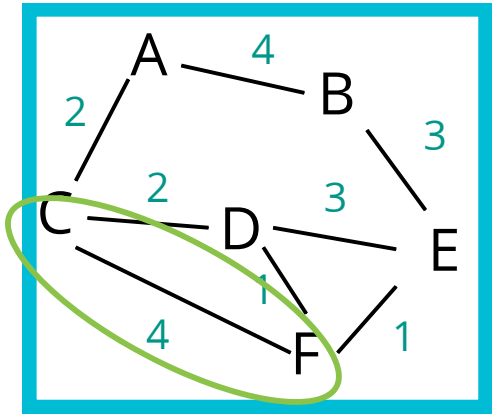
Vertex	Shortest Dist From A
~~A~~	0
B	~~Infinity~~,4
C	~~Infinity~~,2
D	~~Infinity~~,4
E	Infinity
F	Infinity

Visited:

```
[A,C]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: null,
    F: null
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...C

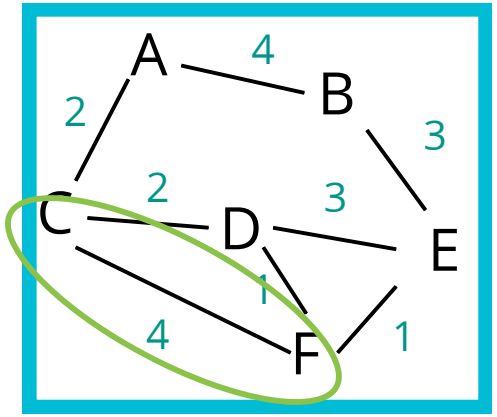
Vertex	Shortest Dist From A
~~A~~	0
B	~~Infinity~~,4
C	~~Infinity~~,2
D	~~Infinity~~,4
E	Infinity
F	Infinity

Visited:

```
[A,C]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: null,
    F: null
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...C

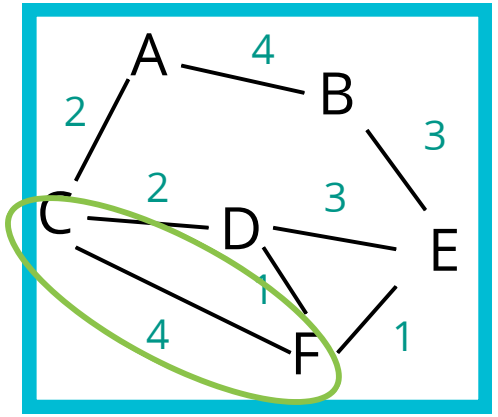
Vertex	Shortest Dist From A
~~A~~	0
B	~~Infinity~~,4
C	~~Infinity~~,2
D	~~Infinity~~,4
E	Infinity
F	~~Infinity~~,6

Visited:

```
[A,C]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: null,
    F: null
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...C

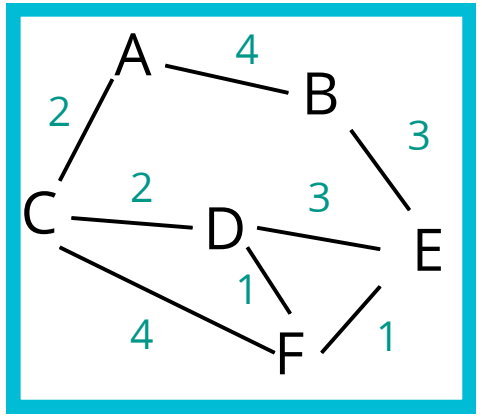
Vertex	Shortest Dist From A
~~A~~	0
B	~~Infinity~~,4
C	~~Infinity~~,2
D	~~Infinity~~,4
E	Infinity
F	~~Infinity~~,6

Visited:

```
[A,C]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: null,
    F: C
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...B

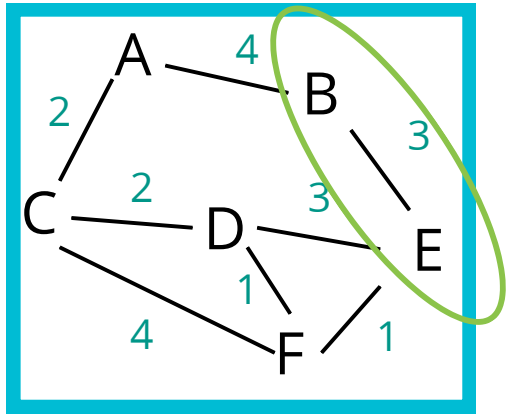
Vertex	Shortest Dist From A
~~A~~	0
B	~~Infinity~~,4
~~C~~	~~Infinity~~,2
D	~~Infinity~~,4
E	Infinity
F	~~Infinity~~,6

Visited:

```
[A,C]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: null,
    F: C
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...B

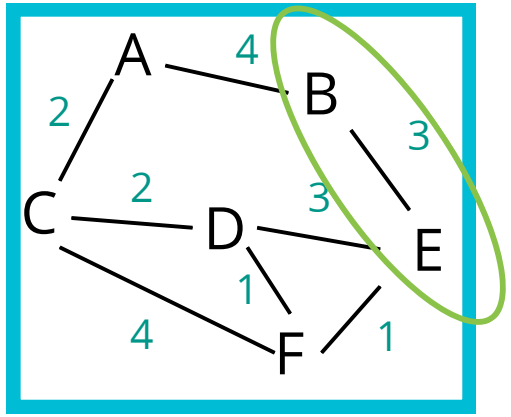
Vertex	Shortest Dist From A
~~A~~	0
B	~~Infinity~~,4
~~C~~	~~Infinity~~,2
D	~~Infinity~~,4
E	Infinity
F	~~Infinity~~,6

Visited:

```
[A,C]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: null,
    F: C
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...B

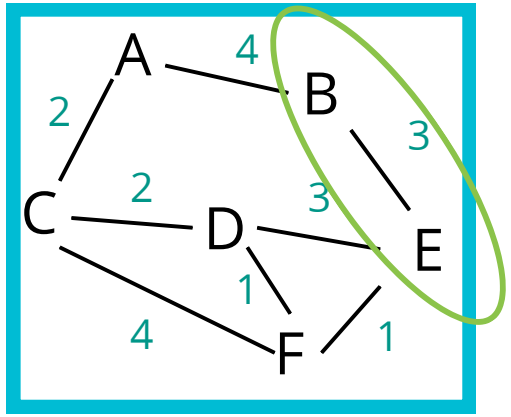
Vertex	Shortest Dist From A
~~A~~	0
B	~~Infinity~~,4
~~C~~	~~Infinity~~,2
D	~~Infinity~~,4
E	~~Infinity~~,7
F	~~Infinity~~,6

Visited:

```
[A,C]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: null,
    F: C
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...B

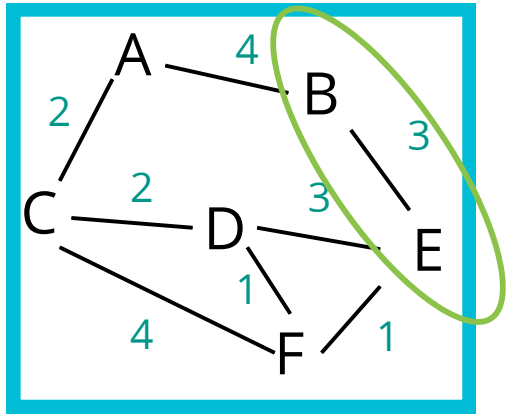
Vertex	Shortest Dist From A
~~A~~	0
B	~~Infinity~~,4
~~C~~	~~Infinity~~,2
D	~~Infinity~~,4
E	~~Infinity~~,7
F	~~Infinity~~,6

Visited:

```
[A,C]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: B,
    F: C
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...B

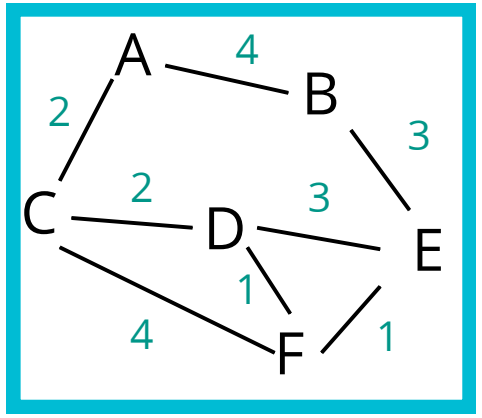
Vertex	Shortest Dist From A
~~A~~	0
B	~~Infinity~~,4
~~C~~	~~Infinity~~,2
D	~~Infinity~~,4
E	~~Infinity~~,7
F	~~Infinity~~,6

Visited:

```
[A,C,B]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: B,
    F: C
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...D

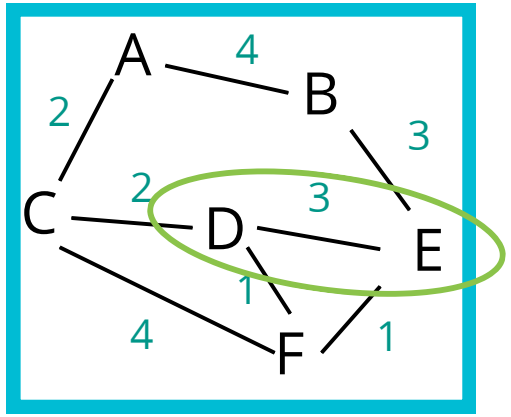
Vertex	Shortest Dist From A
~~A~~	0
~~B~~	~~Infinity~~,4
~~C~~	~~Infinity~~,2
D	~~Infinity~~,4
E	~~Infinity~~,7
F	~~Infinity~~,6

Visited:

```
[A,C,B]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: B,
    F: C
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...D

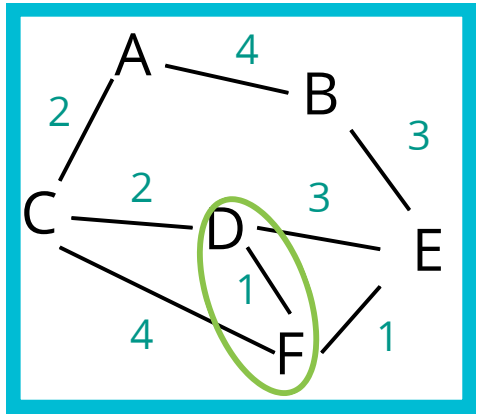
Vertex	Shortest Dist From A
~~A~~	0
~~B~~	~~Infinity~~,4
~~C~~	~~Infinity~~,2
D	~~Infinity~~,4
E	~~Infinity~~,7
F	~~Infinity~~,6

Visited:

```
[A,C,B]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: B,
    F: C
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...D

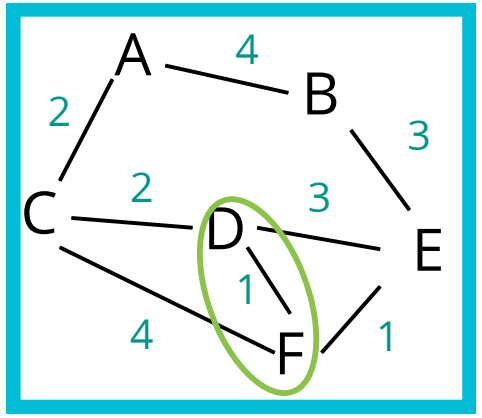
Vertex	Shortest Dist From A
~~A~~	0
~~B~~	~~Infinity~~,4
~~C~~	~~Infinity~~,2
D	~~Infinity~~,4
E	~~Infinity~~,7
F	~~Infinity~~,6

Visited:

```
[A,C,B]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: B,
    F: C
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...D

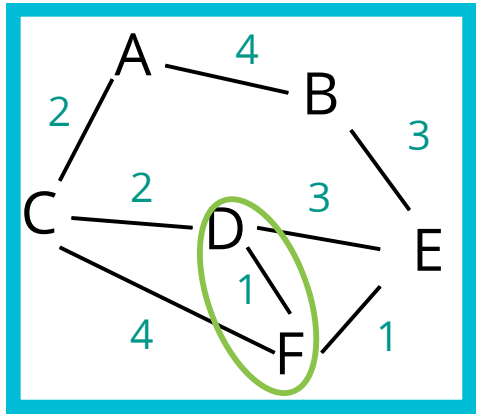
Vertex	Shortest Dist From A
~~A~~	0
~~B~~	~~Infinity~~,4
~~C~~	~~Infinity~~,2
D	~~Infinity~~,4
E	~~Infinity~~,7
F	~~Infinity~~,5

Visited:

```
[A,C,B]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: B,
    F: C
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...D

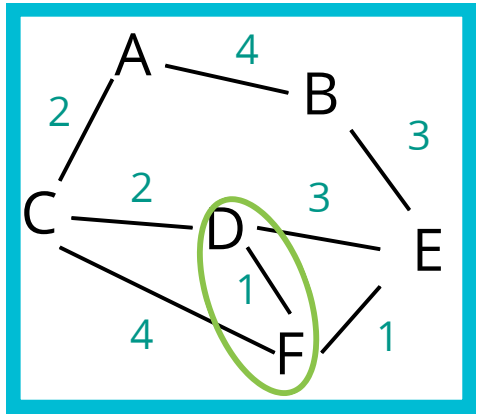
Vertex	Shortest Dist From A
~~A~~	0
~~B~~	~~Infinity~~,4
~~C~~	~~Infinity~~,2
D	~~Infinity~~,4
E	~~Infinity~~,7
F	~~Infinity~~,5

Visited:

```
[A,C,B]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: B,
    F: D
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...D

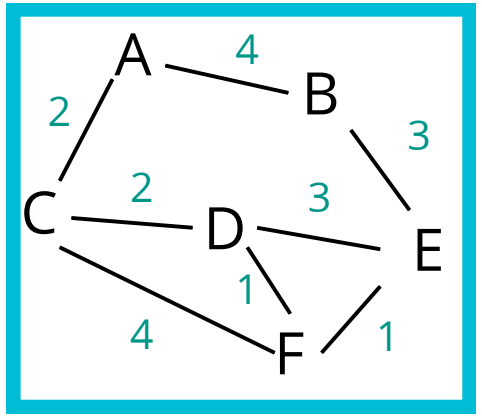
Vertex	Shortest Dist From A
~~A~~	0
~~B~~	~~Infinity~~,4
~~C~~	~~Infinity~~,2
D	~~Infinity~~,4
E	~~Infinity~~,7
F	~~Infinity~~,5

Visited:

```
[A,C,B,D]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: B,
    F: D
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...F

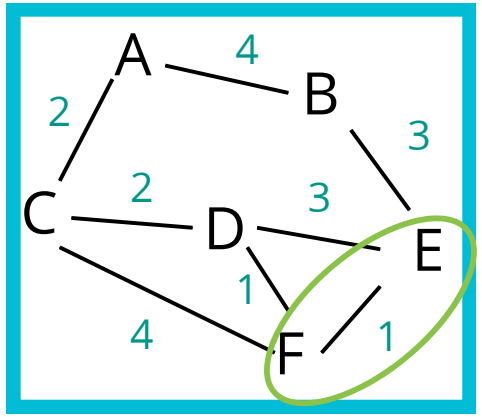
Vertex	Shortest Dist From A
~~A~~	0
~~B~~	~~Infinity~~,4
~~C~~	~~Infinity~~,2
~~D~~	~~Infinity~~,4
E	~~Infinity~~,7
F	~~Infinity~~,5

Visited:

```
[A,C,B,D]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: B,
    F: D
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...F

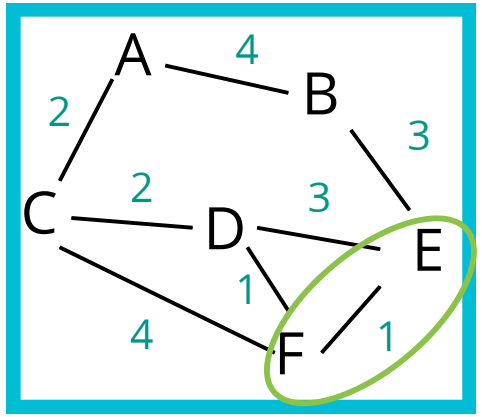
Vertex	Shortest Dist From A
~~A~~	0
~~B~~	~~Infinity~~,4
~~C~~	~~Infinity~~,2
~~D~~	~~Infinity~~,4
E	~~Infinity~~,7
F	~~Infinity~~,5

Visited:

```
[A,C,B,D]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: B,
    F: D
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...F

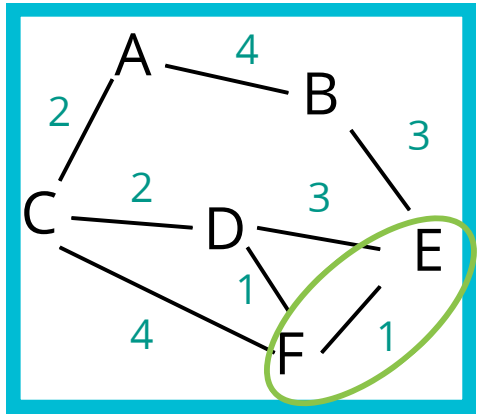
Vertex	Shortest Dist From A
~~A~~	0
~~B~~	~~Infinity~~,4
~~C~~	~~Infinity~~,2
~~D~~	~~Infinity~~,4
E	~~Infinity~~,~~7~~,6
F	~~Infinity~~,5

Visited:

```
[A,C,B,D]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: B,
    F: D
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...F

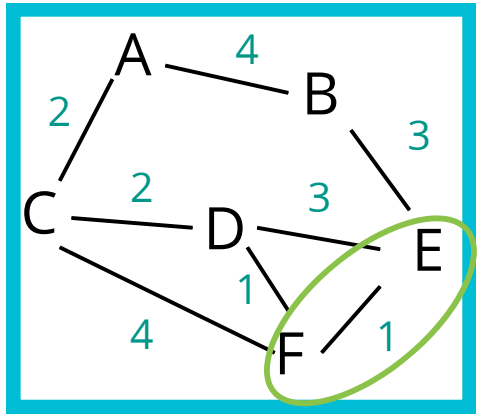
Vertex	Shortest Dist From A
~~A~~	0
~~B~~	~~Infinity~~,4
~~C~~	~~Infinity~~,2
~~D~~	~~Infinity~~,4
E	~~Infinity~~,~~7~~,6
F	~~Infinity~~,5

Visited:

```
[A,C,B,D]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: F,
    F: D
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...F

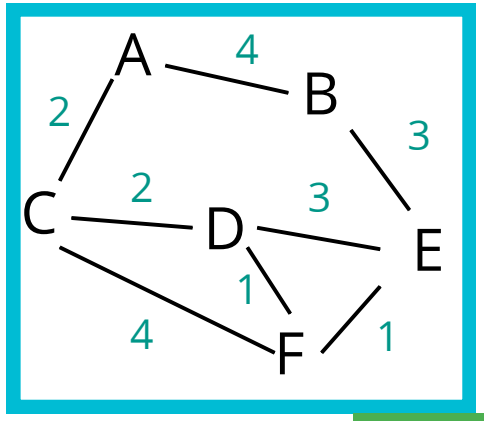
Vertex	Shortest Dist From A
~~A~~	0
~~B~~	~~Infinity~~,4
~~C~~	~~Infinity~~,2
~~D~~	~~Infinity~~,4
E	~~Infinity~~,~~7~~,6
F	~~Infinity~~,5

Visited:

```
[A,C,B,D,F]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: F,
    F: D
}
```



FIND THE SHORTEST PATH FROM A TO E

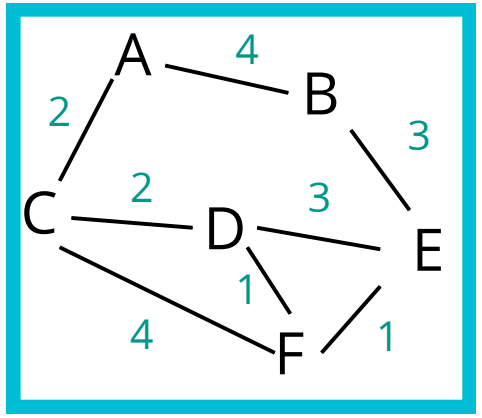
Pick Th

Vertex	S
~~A~~	
~~B~~	
~~C~~	~~Infinity~~,2
~~D~~	~~Infinity~~,4
E	~~Infinity~~,~~7~~,6
~~F~~	~~Infinity~~,5

WE ARE DONE!!!

:

```
    B: A,
    C: A,
    D: C,
    E: F,
    F: D
}
```



FIND THE SHORTEST PATH FROM A TO E

Pick The Smallest...E

Vertex	Shortest Dist From A
~~A~~	0
~~B~~	~~Infinity~~,4
~~C~~	~~Infinity~~,2
~~D~~	~~Infinity~~,4
E	~~Infinity~~,~~7~~,6
~~F~~	~~Infinity~~,5

Visited:

```
[A,C,B,D]
```

Previous:

```
{
    A: null,
    B: A,
    C: A,
    D: C,
    E: F,
    F: D
}
```

A simple PQ

```
class PriorityQueue {
  constructor(){
    this.values = [];
  }
  enqueue(val, priority) {
    this.values.push({val, priority});
    this.sort();
  };
  dequeue() {
    return this.values.shift();
  };
  sort() {
    this.values.sort((a, b) => a.priority - b.priority);
  };
}
```

Notice we are sorting which is O(N * log(N))

Can we do better?

We sure can! Using a min binary heap

Right now, let's focus on Dijkstra's

Dijkstra's Pseudocode

- This function should accept a starting and ending vertex
- Create an object (we'll call it distances) and set each key to be every vertex in the adjacency list with a value of infinity, except for the starting vertex which should have a value of 0.
- After setting a value in the distances object, add each vertex with a priority of Infinity to the priority queue, except the starting vertex, which should have a priority of 0 because that's where we begin.
- Create another object called previous and set each key to be every vertex in the adjacency list with a value of null
- Start looping as long as there is anything in the priority queue
  - dequeue a vertex from the priority queue
  - If that vertex is the same as the ending vertex - we are done!
  - Otherwise loop through each value in the adjacency list at that vertex
    - Calculate the distance to that vertex from the starting vertex
    - if the distance is less than what is currently stored in our distances object
      - update the distances object with new lower distance
      - update the previous object to contain that vertex
      - enqueue the vertex with the total distance from the start node

YOUR TURN

Improving Dijkstra's

Dijkstra's algorithm is greedy! That can cause problems!

We can improve this algorithm by adding a heuristics (a best guess)

Recap

- Graphs are collections of vertices connected by edges
- Graphs can be represented using adjacency lists, adjacency matrices and quite a few other forms.
- Graphs can contain weights and directions as well as cycles
- Just like trees, graphs can be traversed using BFS and DFS
- Shortest path algorithms like Dijkstra can be altered using a heuristic to achieve better results like those with A*